

Introduction to Operating Systems

A computer contains many hardware components such as the processor, memory, disks, keyboard, mouse, display, printers, and network interfaces. Managing all these parts directly is difficult. The **Operating System (OS)** is a special layer of software that makes the computer easier to use and manages all hardware resources efficiently.

What is an Operating System?

An **Operating System** is system software that acts as an interface between the user/application programs and the hardware. It provides a simpler environment for running programs and manages system resources such as CPU, memory, storage, and input/output devices.

Why do we need an Operating System?

A modern computer is a very complex system. It includes:

- One or more processors
- Main memory
- Disks or storage devices
- Keyboard and mouse
- Display/monitor
- Printers
- Network interfaces
- Other input/output devices

If every programmer had to control these devices directly, software development would become extremely difficult. Therefore, the operating system provides:

- a cleaner and simpler model of the computer,
- a way to manage hardware resources,
- a safe environment for running programs,
- efficient coordination among all components.

The OS hides hardware complexity and provides a convenient platform on which application programs can run.

Operating System vs User Interface

Many users think the program they interact with is the operating system. This is not completely correct.

Shell and GUI

The program the user interacts with is usually:

- **Shell** — when the interface is text-based
- **GUI (Graphical User Interface)** — when the interface uses windows, icons, menus, and pointers

Although these programs use the operating system, they are **not the core operating system itself**.

The shell or GUI helps the user interact with the system, but the actual operating system is the lower-level software that controls the hardware and provides services to all programs.

Examples of user programs:

- Web browser
- E-mail reader
- Music player
- Text editor

These programs all depend heavily on the operating system to function properly.

Two Modes of Operation

Most computers operate in two different modes:

1. **Kernel Mode**
2. **User Mode**

Kernel Mode

Kernel Mode (also called **Supervisor Mode**) is the privileged mode in which the operating system runs. In this mode, the OS has complete access to hardware and can execute all machine instructions.

Characteristics of kernel mode:

- Full access to all hardware resources
- Can execute privileged instructions
- Can control memory, CPU, disks, and I/O devices
- Used by the operating system kernel

User Mode

User Mode is the restricted mode in which ordinary application programs run. In this mode, only a subset of machine instructions is allowed.

Characteristics of user mode:

- Limited access to hardware
- Cannot directly perform I/O operations
- Cannot execute privileged instructions
- Must request services from the OS

Comparison

Mode	Kernel Mode	User Mode
Who runs here?	Operating system kernel	Application programs
Hardware access	Full access	Restricted access
Privileged instructions	Allowed	Not allowed
I/O operations	Directly possible	Must go through OS
Purpose	System control and resource management	Running user applications

Why is this separation important?

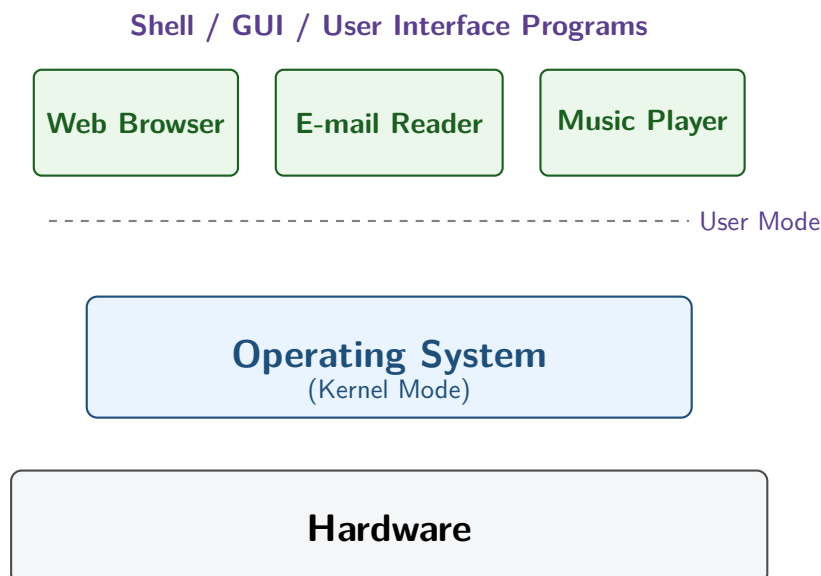
This separation is essential for:

- **Protection** — preventing user programs from damaging the system
- **Security** — preventing unauthorized access to hardware
- **Stability** — preventing one faulty program from crashing everything
- **Control** — ensuring the OS remains in charge of system resources

The OS runs in kernel mode because it must manage and protect the entire system. User applications run in user mode so they cannot directly interfere with critical hardware operations.

Where Does the Operating System Fit?

The operating system sits between hardware and user applications.



Interpretation of the Figure

From bottom to top:

- **Hardware** is the physical part of the computer.
- The **Operating System** runs directly on the hardware.
- **Application Programs** run above the OS in user mode.

- The shell or GUI helps users launch and control these applications.

The operating system provides the foundation on which all other software runs.

OS and Application Programs

There is an important difference between the operating system and normal application software.

Application Programs

A user can:

- install a new browser,
- change the e-mail reader,
- write a new music player,
- replace one application with another.

Operating System Components

A user cannot normally:

- modify the clock interrupt handler,
- rewrite hardware control routines,
- directly change protected kernel functions.

These parts belong to the operating system and are protected by hardware mechanisms.

Application software is usually replaceable by users, but core operating system components are protected and cannot be freely modified.

Boundary of the Operating System

The boundary of the operating system is not always perfectly clear.

Why can the boundary become unclear?

In some systems:

- certain helper programs run in user mode but perform sensitive system tasks,
- parts such as the file system may run in user space,
- embedded systems may not even have a clear kernel/user separation,
- interpreted systems may rely on software rather than hardware for protection.

Example

A password-changing program:

- may run in user mode,
- is not part of the kernel,
- but still performs a sensitive and privileged function.

So, while everything in kernel mode is definitely part of the OS, some user-mode programs may also be closely associated with it.

Characteristics of Operating Systems

Operating systems are very different from ordinary programs.

1. Huge

Modern operating systems are extremely large.

- Core operating systems like Linux or Windows may contain millions of lines of code.
- With libraries included, the total code base can become tens of millions of lines.

2. Complex

An OS must handle:

- process management
- memory management
- file systems
- input/output control

- interrupts
- protection and security
- communication with hardware

3. Long-Lived

Operating systems are not usually discarded and rewritten from scratch. Instead, they evolve gradually over time because:

- they are difficult to design,
- they are costly to replace,
- users need compatibility,
- developers continuously improve them.

Operating systems survive for many years and evolve from version to version rather than being replaced completely.

Examples of Operating System Families

Windows Family

The text mentions that:

- Windows 95 / 98 / Me belonged to one line of development,
- Windows NT / 2000 / XP / Vista / Windows 7 belonged to another.

Even when different versions look similar to users, their internal design may be very different.

UNIX Family

The text also highlights UNIX and its variants:

- System V
- Solaris
- FreeBSD
- Linux

Linux is a separate code base, but it is strongly inspired by UNIX and highly compatible with it.

Operating systems evolve over time. Different versions may appear similar from outside, but their internal architecture can be significantly different.

Conclusion

The operating system is the fundamental software layer of a computer system. It runs directly on hardware, controls and manages system resources, and provides a platform on which all user applications depend. The distinction between kernel mode and user mode is one of the most important concepts in understanding how operating systems work.

The Operating System as an Extended Machine

A computer's hardware is complex and difficult to program directly. The operating system hides this complexity and provides simple, elegant **abstractions** such as files and device interfaces. That is why the OS is called an **extended machine**.

Why is Hardware Difficult to Use?

At the machine level, computer hardware is primitive and awkward. A programmer would otherwise need to handle:

- low-level instructions,
- memory organization,
- I/O mechanisms,
- controller details,
- device-specific operations.

For example, using a modern SATA hard disk directly requires detailed knowledge of hardware behavior, command protocols, and data transfer rules. This is too complex for most application programmers.

Real hardware is powerful, but from a programmer's point of view it is often messy, inconsistent, and inconvenient.

How the OS Solves This Problem

The operating system hides hardware ugliness and provides a cleaner environment for software.

The OS acts as an **extended machine** by presenting a simpler, more usable version of the hardware to programs and programmers.

Its job is not only to control hardware, but also to create useful abstractions and manage them efficiently.

Abstraction Layers

1. Device Drivers

The first layer is the **device driver**.

A device driver is software that communicates directly with a hardware device and hides its low-level details. For example, a disk driver handles the technical details of reading and writing disk blocks.

2. Files

Even device drivers are still too low-level for most applications. So the OS provides a higher abstraction: the **file**.

A **file** is a logical unit of stored information such as a document, photo, song, or email. Programs can create, read, write, and close files without knowing how the disk hardware works.

This is much easier than dealing with controllers, registers, and block operations directly.

The abstraction path is:

Hardware → Device Driver → File / OS Service

Each layer hides more complexity and makes programming easier.

Abstraction: The Central Idea

An **abstraction** is a simplified representation of a complex system that hides unnecessary details and exposes only what is useful.

Abstraction is one of the most important ideas in operating systems because it turns a difficult problem into two manageable ones:

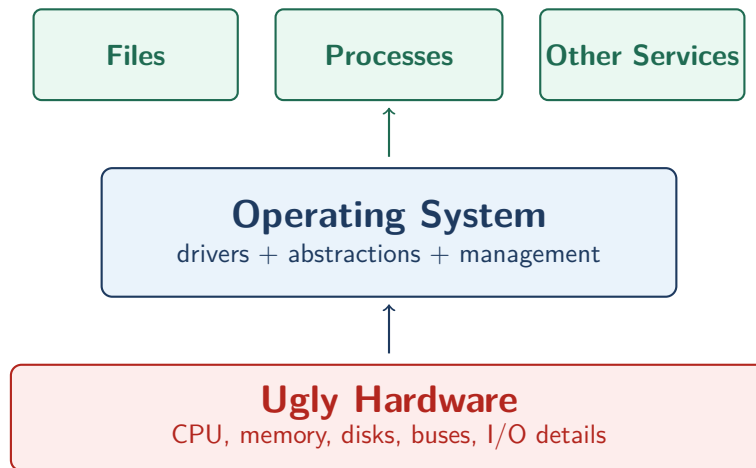
1. designing the abstraction,
2. using the abstraction to solve real problems.

Examples of OS abstractions include:

- files,

- processes,
- virtual memory,
- directories,
- device-independent I/O.

From Ugly Hardware to Beautiful Abstractions



The OS hides hardware complexity and provides elegant abstractions.

Who Uses OS Abstractions?

The real customers of the operating system are mainly:

- application programs,
- application programmers.

These programs directly use OS abstractions such as files, memory services, and process management.

End users usually do **not** interact with the OS directly. They interact through:

- command-line shells,
- graphical desktops,
- menus and windows.

Operating System vs User Interface

The operating system and the user interface are not the same thing.

A system may provide different user interfaces while keeping the same underlying OS abstractions.

Examples:

- Windows desktop and command prompt,
- Linux with Gnome or KDE,
- Linux with a more direct graphical system.

Feature	Operating System	User Interface
Purpose	Provides abstractions and services to programs	Provides interaction for users
Examples	Files, processes, memory, system calls	Shell, desktop, icons, menus
Primary users	Programs and programmers	End users

The Operating System as a Resource Manager

In the **resource manager** view, the operating system is responsible for managing all parts of a computer system and allocating them in an orderly, fair, and controlled way among different programs and users.

Basic Idea

This is a **bottom-up view** of the operating system. Instead of focusing mainly on abstractions, this view focuses on **management of resources**.

Modern computers contain many resources, such as:

- processors,
- main memory,
- timers,
- disks,
- printers,
- network interfaces,
- mice and other I/O devices.

The operating system acts as a **resource manager** by deciding which program gets which resource, when it gets it, and for how long.

Why Resource Management is Necessary

Modern operating systems allow multiple programs to stay in memory and run at the same time. If all of them tried to use the same device directly, the result would be disorder.

For example, if several programs tried to print at the same time on one printer:

- lines from one program could mix with another,
- the output would become confused,
- the printer would not behave in an orderly way.

The operating system prevents this chaos by controlling access to the printer. It may store print output temporarily and send jobs one by one in the correct order.

Without the operating system, shared resources such as the CPU, memory, and printer could easily lead to conflict and confusion.

Main Duties of the OS as a Resource Manager

The operating system must:

- keep track of which program is using which resource,
- grant resource requests,
- delay or reject conflicting requests,
- protect users and programs from interfering with each other,
- account for resource usage,
- ensure fair and efficient sharing.

When multiple users are involved, the need becomes even greater because users may otherwise interfere not only with hardware but also with shared information such as files and databases.

Two Ways of Sharing Resources

Resource management often involves **multiplexing**, which means sharing a resource among many users or programs.

There are two main forms of multiplexing:

1. **Time Multiplexing**
2. **Space Multiplexing**

Time Multiplexing

Time Multiplexing means different programs or users take turns using the same resource over time.

Only one user or program uses the resource at a given moment, but many get a chance by using it one after another.

Example: CPU Sharing

If a system has only one CPU but many programs want to run, the operating system:

- gives the CPU to one program for a short time,
- then switches to another program,
- then to another,
- and later may return to the first one again.

The OS decides:

- who goes next,
- how long each one runs.

Example: Printer Sharing

A printer can also be time multiplexed. If many print jobs are waiting, the operating system decides which job will be printed next.

Time multiplexing means **taking turns**. It is commonly used for the CPU and printers.

Space Multiplexing

Space Multiplexing means dividing a resource into parts so that multiple programs or users can use it at the same time.

Instead of taking turns, each one gets a portion of the resource.

Example: Main Memory

Main memory is usually divided among several running programs. This allows multiple programs to remain in memory at the same time.

This is more efficient than giving all memory to a single program, especially when each program needs only a small part of it.

Example: Disk Space

Disk space is also space multiplexed. A single disk may hold files belonging to many users at once. The operating system must:

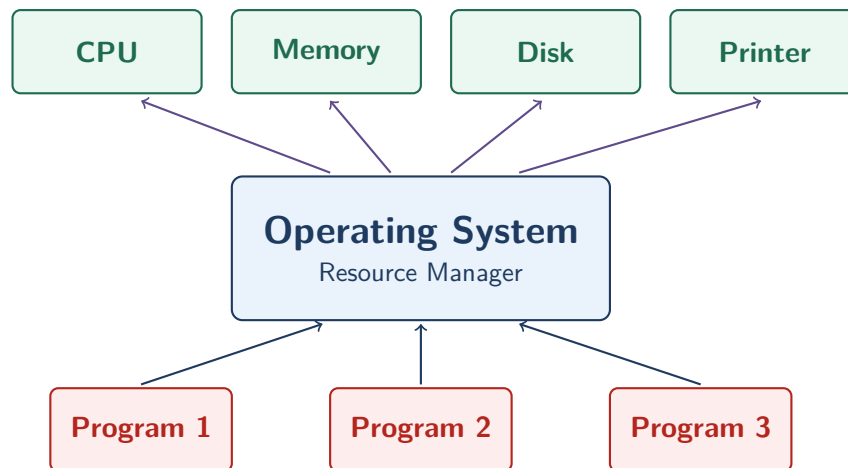
- allocate disk space,
- keep track of used blocks,
- know which user or file owns which part.

Space multiplexing raises issues such as fairness, protection, and tracking. These problems are handled by the operating system.

Time Multiplexing vs Space Multiplexing

Feature	Time Multiplexing	Space Multiplexing
Basic idea	Take turns using the resource	Divide the resource into parts
Usage style	One after another	At the same time
Examples	CPU, printer	Memory, disk
OS decision	Who uses next and for how long	Who gets how much space

Conceptual Diagram



The OS stands between programs and resources and manages their controlled allocation.

Processors

The **CPU (Central Processing Unit)** is the brain of the computer. It executes instructions, manages computation using registers, and works closely with the operating system, especially during context switching, system calls, traps, and protection between user mode and kernel mode.

Basic Role of the CPU

The CPU is responsible for fetching instructions from memory and executing them. Programs run because the CPU repeatedly performs a simple cycle:

1. fetch an instruction from memory,
2. decode the instruction,
3. execute it,
4. move to the next instruction.

This cycle continues until the program finishes.

The repeated **fetch** – **decode** – **execute** cycle is the fundamental working principle of every processor.

Instruction Sets and Architecture

Each CPU can execute only its own specific instruction set. For example:

- an **x86** processor cannot directly execute ARM programs,
- an **ARM** processor cannot directly execute x86 programs.

This means software must match the processor architecture it is built for.

Why registers are needed

Memory access is much slower than CPU execution. To improve performance, CPUs contain fast internal storage locations called **registers**.

Registers are used to hold:

- important variables,

- temporary results,
- addresses,
- control information.

So the instruction set usually includes instructions to:

- load data from memory into a register,
- store data from a register back to memory,
- perform operations using register values.

Important CPU Registers

Besides general-purpose registers, CPUs also have several special registers that are important to both programs and the operating system.

Program Counter (PC)

The **Program Counter** contains the memory address of the next instruction to be fetched.

After an instruction is fetched, the program counter is updated to point to the next one.

Stack Pointer (SP)

The **Stack Pointer** points to the top of the current stack in memory.

The stack contains a frame for each active procedure call. A stack frame typically stores:

- input parameters,
- local variables,
- temporary data not kept in registers.

Program Status Word (PSW)

The **PSW (Program Status Word)** stores condition codes, CPU priority, the current mode (user or kernel), and other control bits.

The PSW is especially important for:

- comparisons,
- system calls,
- interrupt and trap handling,
- I/O control,
- protection.

The operating system must know the contents of all important registers, because it has to save and restore them whenever it switches the CPU from one program to another.

CPU and the Operating System

When multiple programs share the CPU, the operating system performs **time multiplexing**. That means:

- one program runs for some time,
- then the OS stops it,
- then another program runs,
- later the first program may continue again.

To do this correctly, the operating system must save the current program's CPU state, including:

- general registers,
- program counter,
- stack pointer,
- status information.

When the program resumes, those values are restored.

Saving and restoring CPU registers during a switch from one program to another is essential for multitasking.

Pipeline CPU

Modern CPUs do not simply execute one instruction fully before starting the next. To improve speed, they use a **pipeline**.

A **pipeline** divides instruction execution into stages, so different instructions can be in different stages at the same time.

Typical stages are:

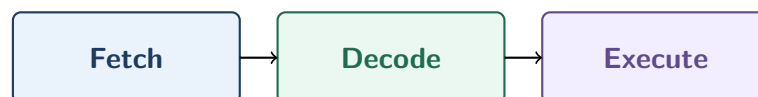
- Fetch
- Decode
- Execute

So while one instruction is being executed:

- the next one may be decoded,
- another one may be fetched.

This increases CPU efficiency.

Simple Pipeline Illustration



Pipelines improve performance, but they also create complications, especially when branches or exceptions occur.

Superscalar CPU

An even more advanced design is the **superscalar CPU**.

A **superscalar CPU** can fetch, decode, and execute multiple instructions at once using multiple execution units.

For example, a CPU may have separate units for:

- integer arithmetic,
- floating-point arithmetic,
- Boolean or logical operations.

Instructions are fetched together, decoded, and placed into a holding buffer. When an execution unit becomes free, it selects a suitable instruction and executes it.

Important consequence

Because of this design:

- instructions may execute **out of order**,
- hardware must ensure the final result remains correct,
- complexity increases for both hardware and the operating system.

CPU Modes: Kernel Mode and User Mode

Most CPUs have two operating modes:

1. **Kernel Mode**
2. **User Mode**

Kernel Mode

In kernel mode:

- the CPU can execute all instructions,
- all hardware features are accessible,
- the operating system normally runs here.

User Mode

In user mode:

- only a subset of instructions can be executed,
- I/O instructions are usually forbidden,
- memory protection features restrict access,
- application programs normally run here.

Kernel mode has full privilege. User mode has limited privilege. This separation protects the system.

System Calls and Trap Mechanism

User programs cannot directly perform privileged operations. To request OS services, they use a **system call**.

A **system call** is a controlled request by a user program to the operating system. It causes a trap into the kernel.

The **TRAP** instruction:

- switches the CPU from user mode to kernel mode,
- transfers control to the operating system,
- allows the OS to perform the requested service.

When the OS finishes, control returns to the user program.

You can think of a system call as:

- a special procedure call,
- plus a mode switch from user mode to kernel mode.

Other Traps and Exceptions

Not all traps come from system calls. Some are caused directly by hardware when exceptional conditions occur, such as:

- division by zero,
- floating-point underflow,
- other exceptional situations.

In every case, the operating system gets control and decides what to do.

Possible actions include:

- terminating the program,
- ignoring or correcting the condition,
- returning control to the program if it has arranged to handle the event.

Traps and exceptions are important because they give the operating system control whenever something unusual or privileged happens.

Multithreaded CPUs

Modern chips often support **multithreading** (also called **hyperthreading** in Intel terminology).

A multithreaded CPU can hold the state of more than one thread and switch between them very quickly.

If one thread is waiting, for example for memory access, the CPU can switch to another thread instead of staying idle.

Important point

Multithreading:

- improves utilization,
- reduces thread-switching delay,
- does **not** provide full parallel execution on a single core.

Only one thread may actually run at a time on that execution path, but switching is very fast.

OS implication

To the operating system, each thread may appear like a separate CPU resource. This can create scheduling issues if the OS places threads inefficiently on the same physical CPU while another CPU stays idle.

Multicore Processors

Beyond multithreading, many chips now contain multiple complete cores.

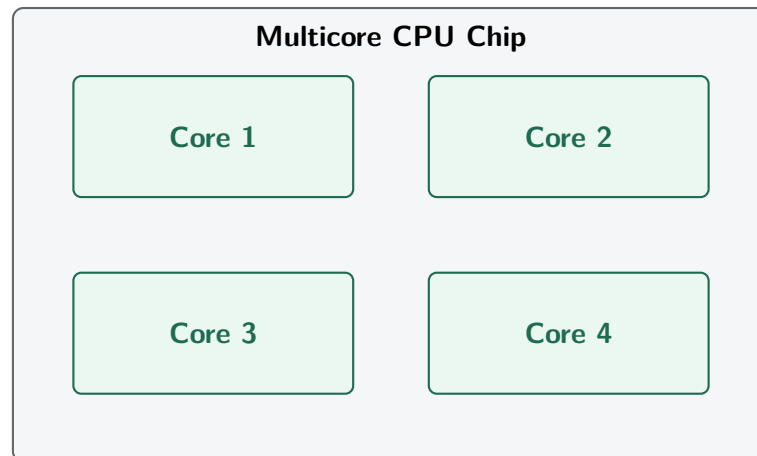
A **multicore processor** has two or more independent CPU cores on a single chip.

Examples:

- quad-core chip = 4 cores,
- many modern chips have 8 or more cores,
- some chips may have dozens of cores.

Each core can execute instructions independently, so multicore chips provide real parallelism.

Simple Multicore Diagram



Multithreading improves switching between threads, while multicore processors provide true parallel execution using multiple cores.

GPU and Many-Core Processing

A **GPU (Graphics Processing Unit)** contains a very large number of small cores.

GPUs are very good for:

- parallel computations,
- graphics rendering,
- workloads with many similar small operations.

They are less suitable for:

- serial tasks,
- complex control-heavy OS logic.

So GPUs may help with special OS-related workloads such as encryption or network processing, but most of the operating system itself typically does not run on the GPU.

Memory

Overview

Memory is the second major component of a computer system. Ideally, memory should be:

- **Very fast** so the CPU does not wait,
- **Very large** so it can store plenty of data,
- **Very cheap** so the system remains affordable.

No current technology can satisfy all three goals at the same time. Therefore, computer systems use a **memory hierarchy**, where different layers provide different trade-offs between **speed**, **capacity**, and **cost per bit**.

1. Memory Hierarchy

Main Idea

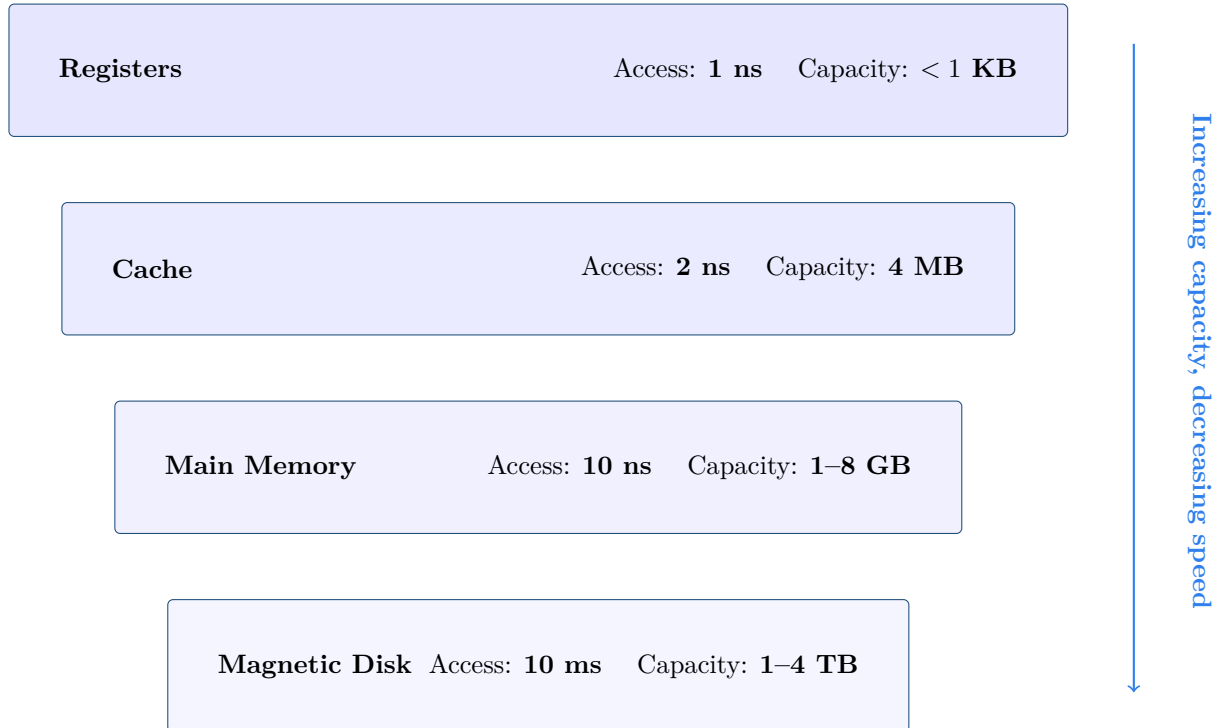
The memory system is arranged in layers. The upper layers are:

- faster,
- smaller in size,
- more expensive per bit.

The lower layers are:

- slower,
- larger in size,
- cheaper per bit.

1.1 Typical Memory Hierarchy Diagram



Observation

As we move downward in the memory hierarchy:

Speed decreases but Capacity increases

This layered design balances overall system performance and cost.

2. Registers

Definition

Registers are the top layer of the memory hierarchy. They are located inside the CPU and are made of the same material as the processor itself. Therefore, they are extremely fast and can be accessed without delay.

Key Features of Registers

- Registers are the **fastest** form of memory.
- Their capacity is **very small**.
- A 32-bit CPU may have around **32 registers of 32 bits each**.
- A 64-bit CPU may have around **64 registers of 64 bits each**.
- Total storage is usually **less than 1 KB**.
- Programs must manage registers in software, deciding which values should remain there.

3. Cache Memory

Definition

Cache memory is a small, high-speed memory placed inside or very close to the CPU. It stores the most frequently used portions of main memory so that the CPU can access data more quickly.

3.1 How Cache Works

Basic Working Principle

Main memory is divided into fixed-size blocks called **cache lines**, usually **64 bytes** each. For example:

- Cache line 0 stores addresses 0–63
- Cache line 1 stores addresses 64–127
- Cache line 2 stores addresses 128–191

When the CPU needs a word from memory, the cache hardware first checks whether the required line is already present in the cache.

3.2 Cache Hit and Cache Miss

Important Terms

- **Cache Hit:** The required data is found in the cache. The request is served immediately.
- **Cache Miss:** The required data is not found in the cache. The system must fetch it from main memory, which is slower.

Performance Note

Cache hits are very fast and may take only about **two clock cycles**. Cache misses are much slower because the system has to access main memory.

3.3 Why Cache is Useful

Caching Beyond RAM

Caching is widely used whenever some items are used more often than others. Examples include:

- Keeping heavily used file data in main memory,
- Storing results of pathname lookups,
- Storing URL-to-IP address conversions,
- Improving operating system performance in many repeated tasks.

3.4 Important Cache Design Questions

Common Questions in Any Caching System

In a caching system, several questions arise:

1. When should a new item be placed into the cache?
2. Which cache line should hold the new item?
3. Which old item should be removed if space is needed?
4. If an item is evicted, where should it be written back in larger memory?

3.5 Address Mapping Example

Example

Suppose:

- there are **4096 cache lines**,
- each line holds **64 bytes**,
- memory addresses are **32 bits**.

Then:

- bits **0–5** may specify the byte inside the cache line,
- bits **6–17** may specify the cache line number.

3.6 Levels of Cache

L1 and L2 Cache

Modern CPUs commonly use multiple levels of cache:

- **L1 Cache:** Smallest and fastest, usually inside the CPU.
- **L2 Cache:** Larger but slightly slower than L1.

Details

- Most CPUs have one L1 cache for instructions and another L1 cache for data.
- Each L1 cache is typically around **16 KB**.
- L2 cache may hold **several megabytes** of recently used memory words.
- L1 cache access is almost immediate.
- L2 cache access may involve a delay of **one or two clock cycles**.

3.7 Cache in Multicore Processors

Shared vs Separate L2 Cache

In multicore chips, designers must decide how the cache is organized:

- **Shared L2 Cache:** One L2 cache is used by all cores.
- **Separate L2 Cache:** Each core has its own L2 cache.

Both approaches have advantages and disadvantages:

- Shared L2 cache needs a more complex controller.
- Separate L2 caches make consistency management more difficult.

4. Main Memory

Definition

Main memory, usually called **RAM (Random Access Memory)**, is the main working memory of the computer. It stores programs and data currently in use by the CPU.

Features of Main Memory

- Main memory is larger than cache but slower.
- It is the **workhorse** of the memory system.
- All CPU requests not satisfied by cache go to main memory.
- Typical size ranges from **hundreds of megabytes** to **several gigabytes**.

Historical Note

Older computers used tiny magnetizable ferrite cores, so main memory was once called **core memory**. The old name still survives in informal usage.

5. Nonvolatile Memory

Meaning

Nonvolatile memory is memory that **does not lose its contents when power is turned off**.

5.1 ROM

ROM: Read Only Memory

- ROM is programmed at the factory.
- Its contents cannot normally be changed afterward.
- It is fast and inexpensive.
- It is often used to store the **bootstrap loader**.
- Some I/O cards also contain ROM for low-level device control.

5.2 EEPROM and Flash Memory

EEPROM and Flash

- **EEPROM** means Electrically Erasable PROM.
- Both EEPROM and flash memory are nonvolatile.
- Unlike ROM, they can be erased and rewritten.
- Writing them is much slower than writing to RAM.
- They are useful because stored programs can be corrected or updated later.

Flash Memory Uses

Flash memory is commonly used in portable electronic devices such as:

- digital cameras,
- portable music players,
- other handheld storage devices.

It is faster than disk but slower than RAM.

Limitation of Flash

Flash memory has a limited number of erase/write cycles. If it is erased too many times, it wears out.

6. CMOS Memory

Definition

CMOS memory is a small, low-power memory used in many computers to store system configuration and time-related information.

Uses of CMOS

CMOS memory is commonly used to store:

- current date and time,
- clock information,
- boot configuration,
- hardware setup parameters.

Why CMOS is Used

CMOS consumes very little power, so it can be supported by a small battery for several years, even when the computer is unplugged.

Battery Failure Effect

When the CMOS battery starts failing, the computer may “forget” important settings such as:

- current time and date,
- which hard disk to boot from,
- stored configuration parameters.

7. Summary Table

Memory Type	Main Characteristics	Volatile / Nonvolatile
Registers	Fastest memory, inside CPU, very small capacity	Volatile
Cache	Very fast, stores frequently used data, improves CPU speed	Volatile
Main Memory (RAM)	Main working memory, larger than cache, stores active programs and data	Volatile
ROM	Factory programmed, cannot usually be changed, used for boot code	Nonvolatile
EEPROM	Electrically erasable and rewritable, slower to write than RAM	Nonvolatile
Flash Memory	Rewritable, portable storage, limited write cycles	Nonvolatile
CMOS	Stores time and configuration using very low battery power	Volatile (battery-backed)

8. Conclusion

Final Note

The memory system of a computer is designed as a hierarchy because no single type of memory can provide maximum **speed**, **capacity**, and **low cost** together. Registers, cache, main memory, ROM, flash memory, and CMOS each serve a unique role. Together, they make the computer system efficient, practical, and cost-effective.

The Operating System Zoo

Overview

Operating systems are developed for different types of computers and different purposes. Some are built for large data centers, some for personal devices, and some for very small or special-purpose systems.

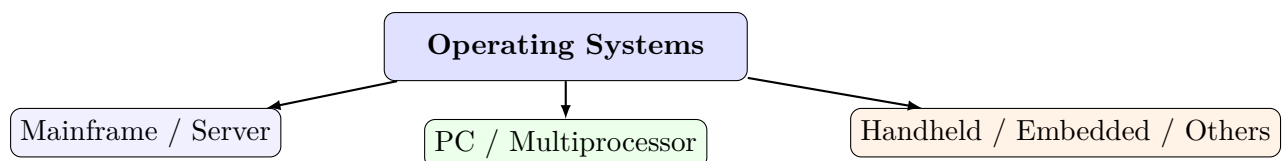
1. Main Types of Operating Systems

Categories

The main categories are:

- Mainframe OS
- Server OS
- Multiprocessor OS
- Personal Computer OS
- Handheld OS
- Embedded OS
- Sensor-Node OS
- Real-Time OS
- Smart Card OS

1.1 Simple Diagram



2. Mainframe Operating Systems

Definition

Mainframe operating systems run on very large computers in corporate data centers. They are designed to handle massive input/output and many jobs at once.

Key Points

- Used in high-end data centers
- Support batch processing
- Support transaction processing
- Support timesharing

Examples

OS/390, Linux, UNIX variants

3. Server Operating Systems

Definition

Server operating systems run on servers that provide services to many users over a network.

Functions

They provide services such as:

- file service
- print service
- web service

Examples

Solaris, FreeBSD, Linux, Windows Server

4. Multiprocessor Operating Systems

Definition

Multiprocessor operating systems are used in systems with multiple CPUs or multicore processors.

Key Points

- Support parallel processing
- Need communication and consistency mechanisms
- Important in modern multicore computers

Examples

Windows, Linux

5. Personal Computer Operating Systems

Definition

Personal computer operating systems are designed mainly for a single user.

Uses

They are used for:

- word processing
- spreadsheets
- games
- Internet access

Examples

Linux, FreeBSD, Windows, OS X

6. Handheld Operating Systems

Definition

Handheld operating systems run on smartphones, tablets, and similar portable devices.

Features

- portable design
- multicore support
- sensors, camera, GPS
- large number of apps

Examples

Android, iOS

7. Embedded Operating Systems

Definition

Embedded operating systems run inside devices like microwaves, TVs, cars, and music players.

Key Idea

These systems usually run fixed software stored in ROM, so users cannot normally install new applications.

Examples

Embedded Linux, QNX, VxWorks

8. Sensor-Node Operating Systems

Definition

Sensor-node operating systems run on tiny wireless sensor computers.

Characteristics

- battery-powered
- limited RAM
- event-driven
- designed for long unattended use

Example

TinyOS

9. Real-Time Operating Systems

Definition

Real-time operating systems are designed for systems where timing is critical.

Types

- **Hard real-time:** deadlines must always be met
- **Soft real-time:** occasional delay is acceptable

Examples

Hard real-time: industrial control, avionics
Soft real-time: multimedia, smartphones
OS example: eCos

10. Smart Card Operating Systems

Definition

Smart card operating systems run on tiny card-sized devices with very limited memory and power.

Key Points

- very small and simple
- some support only one function
- some can run Java applets
- need scheduling and protection for multiple applets

11. Conclusion

Summary

The operating system zoo includes many different types of systems. Each type is designed according to hardware size, user needs, and application purpose.

Processes

Definition

A **process** is a program in execution. It is one of the most important concepts in operating systems. A process is not just the program code; it also includes memory, resources, and all information needed to run that program.

1. What a Process Contains

Main Components

Each process generally contains:

- an **address space**,
- the **executable program**,
- the program's **data**,
- the **stack**,
- **register values**,
- **open files**,
- **alarms and signals**,
- information about **related processes**.

Address Space

The **address space** is the range of memory locations a process can use. It contains the program code, data, and stack.

2. Process in a Multiprogramming System

Idea

In a multiprogramming system, several processes may exist at the same time. For example, a user may run a video editor, a web browser, and an email receiver simultaneously.

CPU Switching

The operating system does not run all processes at the exact same instant on one CPU. Instead, it switches the CPU from one process to another. This gives the appearance that many programs are running together.

3. Suspending and Restarting a Process

Saving Process State

When a process is temporarily stopped, it must later continue in exactly the same state. So the operating system must save all important information before suspension.

Saved Information

The following information may need to be saved:

- register contents,
- program counter,
- stack pointer,
- open file information,
- current file positions,
- other execution details.

Process Table

In many operating systems, information about each process is stored in a **process table**. It is usually an array of entries, with one entry for each active process.

Core Image

A suspended process consists of:

- its **address space** or **core image**,
- its **process table entry**.

4. Process Creation and Termination

Basic System Calls

Important process-management system calls deal with:

- creating a process,
- terminating a process,
- waiting for a child process,
- requesting or releasing memory,
- replacing a program with another one.

Example

A command interpreter or **shell** reads user commands. If the user asks to compile a program, the shell creates a new process to run the compiler. After finishing, that process terminates itself.

5. Parent and Child Processes

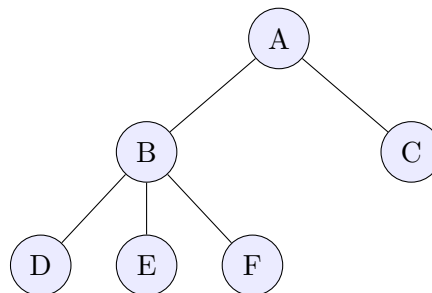
Relationship

A process can create another process. The creating process is called the **parent process**, and the new one is called the **child process**.

Process Tree

If child processes create more child processes, a **process tree** is formed.

5.1 Simple Process Tree



Explanation

Here:

- Process A creates B and C
- Process B creates D, E, and F

6. Interprocess Communication

Meaning

When related processes work together, they often need to exchange information and coordinate their activities. This is called **interprocess communication (IPC)**.

Purpose of IPC

IPC is needed for:

- data sharing,
- synchronization,
- cooperation between related processes.

7. Signals and Alarms

Signal

A **signal** is a notification sent to a process. It causes the process to temporarily stop its current activity and run a special signal-handling routine.

Alarm Signal

A process may request the operating system to send it an alarm after a certain number of seconds. This is useful when the process needs a timeout mechanism.

Example

If a process sends a message over a network and no reply arrives in time, an alarm signal may be used to trigger retransmission.

Important Note

Signals are the software equivalent of hardware interrupts. They may occur because of:

- timer expiration,
- illegal instruction,
- invalid memory address,
- other exceptional events.

8. User IDs and Group IDs

UID and GID

Each authorized user in a system is assigned a **UID** (User Identification). Users may also belong to groups, each identified by a **GID** (Group Identification).

Key Ideas

- Every process has the UID of the user who started it.
- A child process usually has the same UID as its parent.
- Group membership helps control access and permissions.

9. Superuser / Administrator

Special User

One special user has extra privileges:

- **Superuser** in UNIX
- **Administrator** in Windows

This user can override many protection rules.

Note

In large systems, only the system administrator should know the password for this powerful account.

10. Conclusion

Summary

A process is the basic unit of execution in an operating system. It includes the program, memory, resources, and control information needed for execution. Process creation, suspension, communication, and protection are central parts of operating system design.

Address Spaces

Definition

An **address space** is the set of memory addresses a process is allowed to use. It is one of the most important concepts in memory management of an operating system.

1. Basic Idea

Main Concept

Every computer has **main memory** for holding executing programs. In a very simple operating system, only **one program** can stay in memory at a time. If another program has to run, the first one must be removed.

Need for Protection

Modern operating systems allow **multiple programs** to stay in memory at the same time. To prevent them from interfering with each other and with the operating system, a **protection mechanism** is required.

2. Memory Protection

Protection Mechanism

The protection mechanism is implemented in **hardware**, but it is **controlled by the operating system**. This ensures that one process cannot wrongly access the memory of another process.

Why Protection is Needed

Without protection:

- one program may overwrite another program,
- one process may damage the operating system,
- the system may become unstable or crash.

3. Address Space of a Process

Process View

Normally, each process has its own range of addresses, usually starting from **0** up to some **maximum value**. The process can use only addresses inside this range.

Simple Case

In the simplest case:

- the process address space is smaller than main memory,
- the whole process can fit in physical memory,
- execution becomes straightforward.

4. Problem of Large Address Space

Modern Computers

Modern computers commonly use **32-bit** or **64-bit** addresses.

$$2^{32} \text{ bytes} \quad \text{or} \quad 2^{64} \text{ bytes}$$

This gives a very large possible address space.

The Problem

Sometimes a process may have an address space larger than the available main memory. Then the whole process cannot be kept in RAM at the same time.

Old Situation

In early computers, if a process needed more address space than physical memory, it simply could not run properly.

5. Virtual Memory

Definition

Virtual memory is a technique in which the operating system keeps part of a process's address space in **main memory** and part on **disk**, and moves pieces back and forth when needed.

Main Idea of Virtual Memory

Virtual memory creates the **abstraction** that a process has a large continuous address space, even if all of it is not physically present in RAM at one time.

How It Works

The operating system:

- keeps some parts of the address space in RAM,
- keeps the remaining parts on disk,
- transfers data between RAM and disk as required.

6. Address Space vs Physical Memory

Important Difference

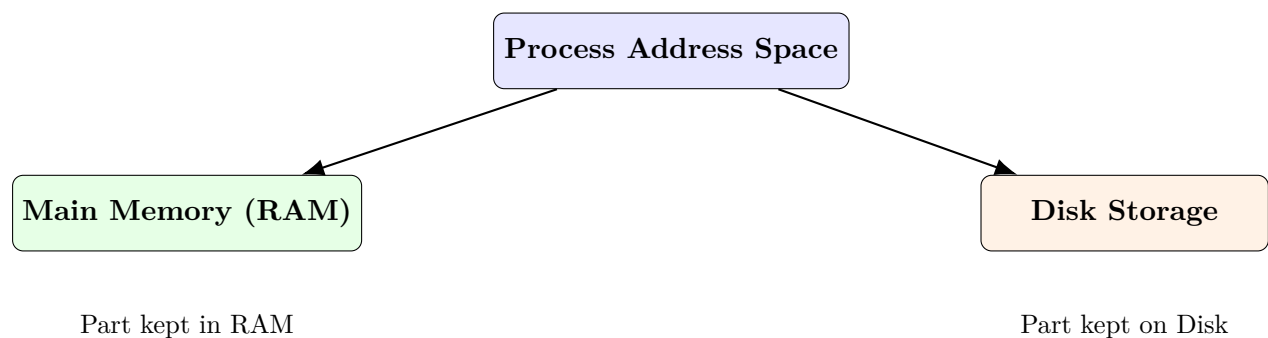
The **address space** of a process is a logical view, while **physical memory** is the actual RAM installed in the machine.

Key Observation

The address space is **decoupled** from physical memory. Therefore, it may be:

- **larger** than physical memory, or
- **smaller** than physical memory.

6.1 Simple Diagram



7. Role of the Operating System

OS Responsibility

Managing **address spaces** and **physical memory** is a major responsibility of the operating system.

Main Tasks of OS

The operating system must:

- manage memory allocation,
- protect processes from one another,
- map address space to physical memory,
- move data between RAM and disk when necessary.

Files

Definition

A **file system** is a key concept in almost all operating systems. Its main job is to hide the hardware details of disks and other I/O devices and present the user with a clean, device-independent way of storing and accessing data as **files**. Based on user text
:contentReference[oaicite:0]index=0

1. Basic File Operations

Common System Calls

The operating system provides system calls to:

- create a file,
- remove a file,
- read a file,
- write a file,
- open a file,
- close a file.

Before a file is read or written, it must usually be **opened**. After use, it should be **closed**.

2. Directories

Directory

A **directory** is a way of grouping related files together. Operating systems use directories to organize files in a structured manner.

Why Directories are Useful

A user may keep:

- one directory for course materials,
- one directory for email,
- one directory for web pages,
- one directory for personal documents.

Directory Operations

The operating system provides system calls to:

- create directories,
- remove directories,
- place files inside directories,
- remove files from directories.

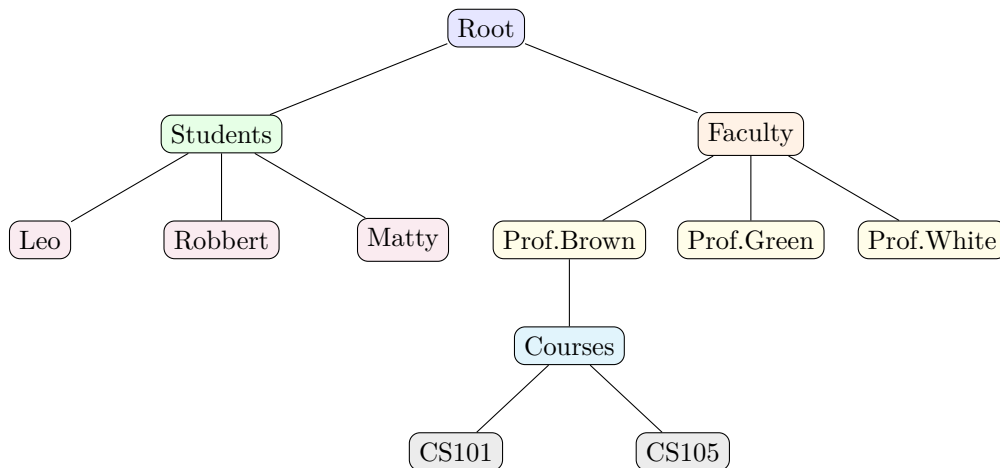
Directory entries may be either **files** or **other directories**.

3. File-System Hierarchy

Tree Structure

Directories and files form a **hierarchy**, often organized as a tree. At the top is the **root directory**.

3.1 Simple File Hierarchy Diagram



Note

File-system hierarchies are usually much deeper and longer-lived than process hierarchies. A directory tree may remain in use for years.

4. Path Names

Absolute Path Name

Every file can be identified by giving its **path name** from the root directory. Such a path is called an **absolute path name**.

Example

An example of an absolute UNIX path is:

/Faculty/Prof.Brown/Courses/CS101

The leading slash “/” means the path starts from the root directory.

UNIX vs Windows

- In **UNIX**, the separator is /
- In **Windows**, the separator is \

So the same path in Windows style would be:

\Faculty\Prof.Brown\Courses\CS101

5. Current Working Directory

Working Directory

Each process has a **current working directory**. If a path does not begin with a slash, the operating system searches for it relative to the current working directory.

Example

If the current working directory is:

/Faculty/Prof.Brown

then the relative path

Courses/CS101

refers to the same file as the absolute path:

/Faculty/Prof.Brown/Courses/CS101

Changing Directory

A process can change its working directory by using a system call.

6. Opening Files and File Descriptors

Open and Permissions

Before a file can be read or written, it must be **opened**. At that time, the operating system checks whether access is allowed.

File Descriptor

If access is allowed, the operating system returns a small integer called a **file descriptor**. This file descriptor is then used in later operations like reading and writing.

Access Failure

If permission is denied, the operating system returns an **error code**.

7. Mounted File System

Mounted File System

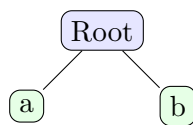
A **mounted file system** allows a separate file system, such as one on a CD-ROM, USB drive, or external disk, to be attached to the main directory tree.

Main Idea

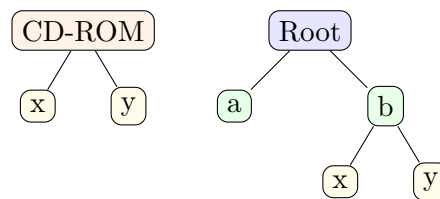
Instead of using device names in file paths, UNIX attaches the external file system to an existing directory in the main tree. This is called **mounting**.

7.1 Simple Mount Diagram

(a) Before Mounting



(b) After Mounting



Example

If a CD-ROM file system is mounted on directory `/b`, then its files may be accessed as:

`/b/x` and `/b/y`

8. Special Files

Special File

UNIX uses **special files** to make I/O devices appear like files. This allows devices to be accessed using ordinary file operations such as read and write.

Types of Special Files

There are two main types:

- **Block special files**
- **Character special files**

Block vs Character

- **Block special files** represent devices with randomly addressable blocks, such as disks.
- **Character special files** represent devices that handle character streams, such as printers and modems.

Example

By convention, special files are stored in the `/dev` directory.

Example:

`/dev/lp`

may represent a printer.

9. Pipes

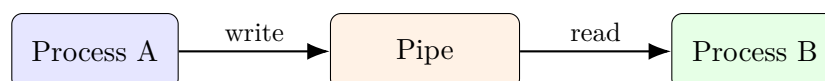
Pipe

A **pipe** is a pseudofile used to connect two processes. It allows one process to send data to another in a file-like manner.

How a Pipe Works

- Process A writes data to the pipe.
- Process B reads data from the pipe.
- The interaction looks similar to file writing and file reading.

9.1 Pipe Diagram



Key Idea

In UNIX, communication through pipes looks very similar to ordinary file I/O. A process may not even know whether it is writing to a real file or to a pipe unless it checks specially.

Input / Output

Definition

Input/Output (I/O) refers to the process of taking input into a computer and producing output from it. Input devices allow users to give instructions to the computer, and output devices allow the computer to show the results.

1. Basic Idea

Main Concept

Every computer has physical devices for:

- acquiring **input**,
- producing **output**.

Without input and output devices, a computer would not be useful because users could neither give commands nor receive results.

2. Examples of I/O Devices

Common Devices

Many kinds of input and output devices exist. Some common examples are:

- **Keyboard** — input device
- **Monitor** — output device
- **Printer** — output device

More Examples

Other I/O devices may include:

- mouse,
- scanner,
- speakers,
- microphone,
- touchscreen.

3. Role of the Operating System

OS Responsibility

It is the responsibility of the **operating system** to manage all input and output devices.

Why OS Manages I/O

The operating system controls I/O devices so that:

- users can interact with the computer easily,
- programs can use devices without dealing with hardware details,
- the system works efficiently and correctly.

4. I/O Subsystem

Definition

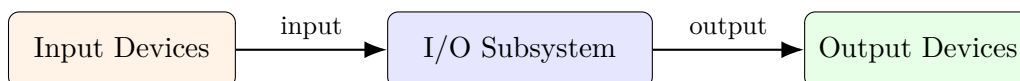
Every operating system contains an **I/O subsystem**. This subsystem is responsible for managing all I/O operations and devices.

Main Function

The I/O subsystem acts as a bridge between:

- the **hardware devices**, and
- the **operating system / user programs**.

4.1 Simple I/O Diagram



5. Device-Independent I/O Software

Meaning

Some parts of I/O software are **device independent**. This means they are general and can work with many or all I/O devices.

Importance

Device-independent software helps the operating system provide a common method of handling different devices, instead of writing everything separately for each one.

6. Device Drivers

Definition

A **device driver** is a special part of the I/O software that is designed for a specific hardware device.

Key Idea

Unlike device-independent software, device drivers are:

- device specific,
- written for particular hardware,
- responsible for controlling that device directly.

Examples

Different devices need different drivers, for example:

- keyboard driver,
- printer driver,
- display driver.

7. Device-Independent Software vs Device Drivers

Feature	Device-Independent I/O Software	Device Drivers
Nature	General purpose	Device specific
Use	Works for many devices	Works for one particular device
Role	Provides common I/O handling	Controls hardware directly
Examples	General I/O services	Keyboard, printer, monitor drivers

Protection

Definition

Protection means keeping computer information and system resources safe from unauthorized access, misuse, or damage. It is one of the important duties of the operating system.

1. Why Protection is Needed

Main Idea

Computers store a huge amount of important and confidential information. Users often want this information to remain private and secure.

Examples of Sensitive Information

Protected information may include:

- email,
- business plans,
- tax returns,
- personal files,
- official documents.

Role of the Operating System

It is the responsibility of the **operating system** to manage system security so that files and other resources are accessible only to authorized users.

2. File Protection in UNIX

UNIX Protection Code

In UNIX, each file is protected by a **9-bit binary protection code**. This code determines who can read, write, or execute the file.

Three 3-Bit Fields

The 9 bits are divided into three groups:

- one field for the **owner**,
- one field for the **group**,
- one field for **others**.

3. The rwx Bits

Meaning of rwx

Each 3-bit field contains:

- **r** = read permission
- **w** = write permission
- **x** = execute permission

These are called the **rwx bits**.

Permission Groups

So permissions are checked separately for:

- the file owner,
- members of the owner's group,
- all other users.

4. Example of UNIX Protection Code

Example

Consider the protection code:

`rwxr-x-x`

This means:

- **Owner:** `rw` → read, write, execute
- **Group:** `r-x` → read, execute only
- **Others:** `-x` → execute only

Meaning of Dash

A dash - means that the corresponding permission is **absent**.

5. Protection of Directories

Directory Permission

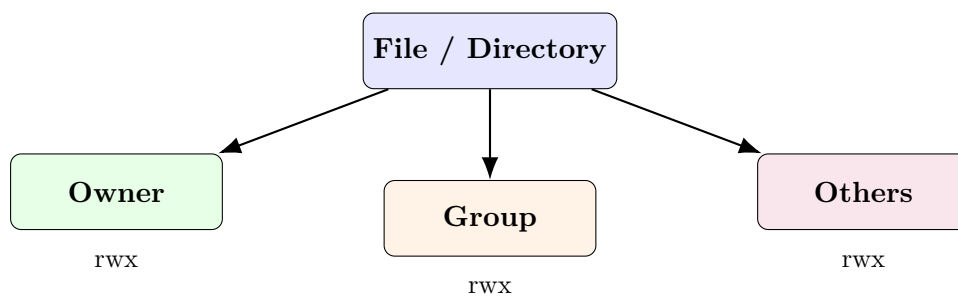
For a **directory**, the **x** bit means **search permission**. It allows a user to search or move through that directory.

Directory Permissions

Thus, directory permissions help control:

- who can view directory contents,
- who can modify files in it,
- who can search through it.

6. Simple Permission Diagram



7. Security Beyond File Protection

Broader Security

File protection is only one part of operating-system security. There are many other protection issues as well.

Other Security Threats

The operating system must also protect the system from:

- unwanted human intruders,
- malicious software,
- viruses,
- unauthorized access attempts.

8. Why OS Security Matters

Importance

Good protection is necessary to:

- keep user data confidential,
- prevent accidental damage,
- stop unauthorized access,
- keep the system reliable and safe.

The Shell

Definition

The **shell** is a command interpreter used in UNIX. It is **not part of the operating system itself**, but it uses many operating-system features and acts as an important interface between the user and the operating system.

1. Shell and Operating System

Important Idea

The operating system is the code that carries out **system calls**. Programs such as:

- editors,
- compilers,
- assemblers,
- linkers,
- utility programs,
- command interpreters

are useful, but they are **not part of the operating system**.

Why the Shell Matters

Although the shell is not part of the operating system, it makes heavy use of operating-system features. That is why it is a very good example of how system calls are used in practice.

2. What the Shell Does

Main Function

The shell is the main interface between:

- the **user**, and
- the **operating system**

unless the user is working through a graphical user interface (GUI).

Common UNIX Shells

Different UNIX shells exist, such as:

- sh
- csh
- ksh
- bash

All of them support the basic shell functionality.

3. How the Shell Starts

Login

When a user logs in, a shell is started automatically. The shell uses the terminal as:

- **standard input**
- **standard output**

Prompt

The shell begins by showing a **prompt**, often a dollar sign (\$). The prompt tells the user that the shell is ready to accept a command.

4. Running a Command

Example

If the user types:

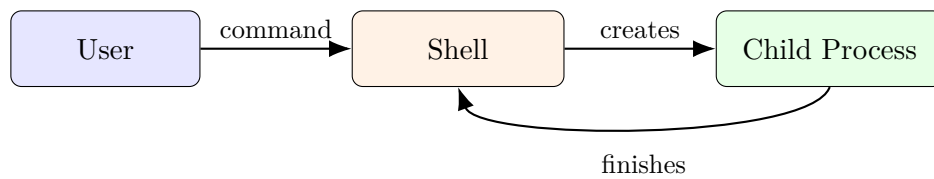
date

the shell creates a **child process** and runs the **date** program in that child.

What Happens Next

- The shell creates a child process.
- The child runs the requested program.
- The shell waits for the child to finish.
- After termination, the shell prints the prompt again.

4.1 Simple Command Flow Diagram



5. Output Redirection

Redirection

The user can redirect standard output to a file instead of sending it to the terminal.

Example

```
date >file
```

This sends the output of `date` to `file`.

6. Input and Output Redirection

Input + Output

Standard input can be taken from a file, and standard output can also be sent to another file.

Example

```
sort <file1 >file2
```

Here:

- `sort` takes input from `file1`
- the sorted result goes to `file2`

7. Pipes

Pipe

A **pipe** allows the output of one program to become the input of another program.

Example

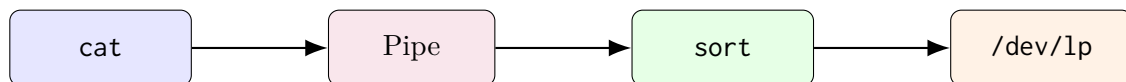
```
cat file1 file2 file3 | sort >/dev/lp
```

Explanation

In this command:

- `cat` combines the contents of three files,
- the output is passed through a pipe to `sort`,
- `sort` arranges the lines alphabetically,
- the final output is sent to `/dev/lp`, usually the printer.

7.1 Pipe Diagram



8. Background Jobs

Ampersand

If a command ends with an ampersand (`&`), the shell does **not wait** for that command to complete. Instead, it returns the prompt immediately.

Example

```
cat file1 file2 file3 | sort >/dev/lp &
```

This command runs as a **background job**.

Meaning

A background job allows the user to continue doing other work while the command is still running.

9. Shell vs GUI

Graphical User Interface

Most personal computers today use a **GUI** (Graphical User Interface). A GUI is also just a program running on top of the operating system, similar to the shell.

Examples

In Linux, users may choose:

- Gnome,
- KDE,
- or no GUI at all.

Key Idea

Both the shell and the GUI are interfaces for users. Neither of them is the operating system itself; both run on top of the operating system.

System Calls

Definition

System calls are the main interface between user programs and the operating system. Through system calls, programs request services such as reading files, creating processes, writing data, and performing input/output.

1. Main Role of System Calls

Basic Idea

Operating systems have two major functions:

- providing **abstractions** to user programs,
- managing the computer's **resources**.

Most interaction between user programs and the operating system happens through system calls dealing with abstractions such as files and processes.

Key Observation

The resource-management part of the operating system is mostly transparent to users. But system calls give users and programs access to operating-system services in a controlled way.

2. System-Call Interface

Interface

To understand what an operating system really does, we must study the interface between user programs and the operating system. This interface is formed mainly by **system calls**.

Variation

System calls differ from one operating system to another. However, the main ideas are usually very similar across modern systems.

3. POSIX and UNIX Style

POSIX

Many modern operating systems follow the **POSIX** style of system calls. This includes UNIX, System V, BSD, Linux, MINIX 3, and similar systems.

Important Note

Although details differ from one system to another, most modern operating systems provide system calls that perform similar jobs.

4. Library Procedures and System Calls

Why Library Procedures Are Used

The actual mechanism of making a system call is machine dependent and often written in assembly language. So a **procedure library** is provided to allow programmers to make system calls from C and other languages.

Idea

A user program usually does not invoke the kernel directly. Instead:

- the program calls a library function,
- the library function performs the actual system call,
- the operating system carries out the requested service.

5. User Mode and Kernel Mode

Mode Switching

A single-CPU computer can execute only one instruction at a time. When a user program needs an operating-system service, it executes a **trap instruction** to transfer control from **user mode** to **kernel mode**.

Why Trap is Needed

A system call is like a special procedure call, but unlike an ordinary procedure call, it enters the **kernel**.

6. Example: read System Call

read

A commonly used system call is **read**. It is used to read data from a file or input source.

C Example

```
count = read(fd, buffer, nbytes);
```


Parameters of read

The read call has three parameters:

- `fd` — file descriptor specifying the file,
- `buffer` — memory area where data will be stored,
- `nbytes` — number of bytes requested.

Return Value

The call returns the number of bytes actually read. Usually this equals `nbytes`, but it may be smaller, for example at end-of-file.

Error Case

If the system call fails because of an invalid parameter or disk error:

- the return value becomes `-1`,
- the error number is placed in the global variable `errno`.

Programs should always check the result of a system call.

7. How a System Call Works

General Steps

System calls are performed through a sequence of steps involving the user program, library procedure, trap instruction, kernel, and return path.

Main Steps

For a call like `read(fd, buffer, nbytes)`:

1. parameters are prepared,
2. library procedure is called,
3. system-call number is placed in a register,
4. TRAP instruction switches to kernel mode,
5. kernel checks the system-call number,
6. correct handler is executed,
7. control returns back to user space.

8. System Call Mechanism

System Call Steps

A system call such as `read(fd, buffer, nbytes)` goes through several stages: parameter passing, library call, trap to kernel, dispatching, handler execution, and return to user space.

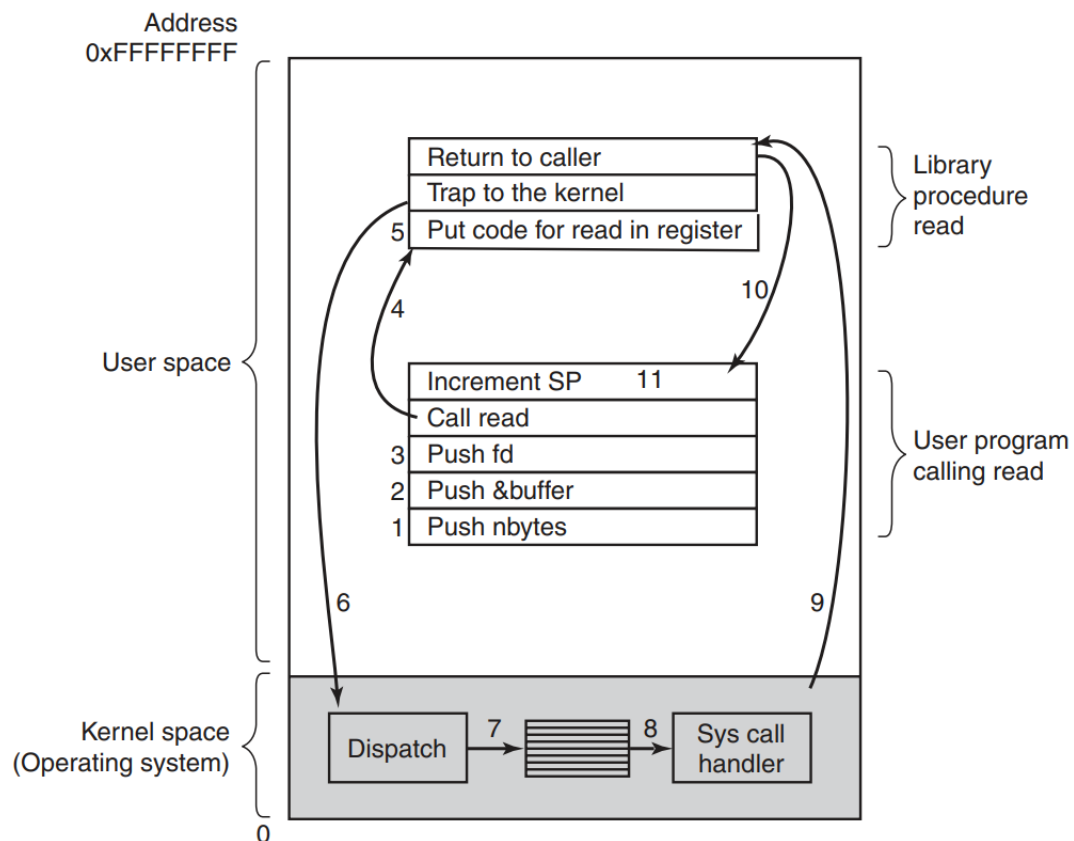


Figure 1: The 11 steps in making the system call `read(fd, buffer, nbytes)`.

9. Stack and Parameter Passing

Stack Use

Before calling the library procedure, the program places parameters on the stack. In C and C++, parameters are usually pushed in reverse order.

By Value and By Reference

In the example:

- the first and third parameters are passed **by value**,
- the second parameter is passed **by reference**,
- this means the address of the buffer is passed, not its contents.

10. TRAP Instruction

Trap

The **TRAP** instruction is used to switch from user mode to kernel mode and start execution inside the operating system.

How Trap Differs from Procedure Call

TRAP is similar to a procedure call, but it differs in two important ways:

- it switches the CPU into **kernel mode**,
- it does not jump to an arbitrary address like a normal procedure call.

11. System-Call Handler

Handler

After entering the kernel, the operating system examines the **system-call number** and dispatches control to the correct **system-call handler**.

Dispatching

This is usually done with a table of pointers to handlers, indexed by system-call number.

12. Returning from the System Call

Return Path

After the system-call handler finishes its work, control returns to the library procedure and then back to the user program.

Final Cleanup

After the return, the user program removes the parameters from the stack and continues execution.

13. Blocking System Calls

Blocking

Sometimes a system call cannot complete immediately. In such a case, the calling process may be **blocked**.

Example

If a process tries to read from the keyboard and no character has been typed yet, the process must wait. The operating system may then run another process until the required input becomes available.

Meaning

So a system call does not always return immediately. It may suspend the process until the requested event or data is ready.

14. Services Provided by System Calls

What System Calls Do

System calls provide the main operating-system services required by programs.

Examples of Services

These services include:

- creating and terminating processes,
- creating and deleting files,
- reading and writing files,
- managing directories,
- performing input and output.

15. POSIX Procedures vs System Calls

Not Always One-to-One

The mapping between POSIX procedure calls and actual system calls is not always one-to-one.

Important Idea

- Some POSIX procedures may be handled completely in user space.
- Some procedures directly invoke one system call.
- In some cases, one system call may handle several related library procedures.

Reason

If an operation can be done without entering the kernel, it is usually done in user space for better performance.

16. Conclusion

Summary

System calls are the fundamental bridge between user programs and the operating system. They allow programs to request kernel services in a safe and controlled way. By studying system calls, we understand how the operating system provides abstractions such as files, processes, and input/output to user programs.

System Calls for Process Management

Definition

Process-management system calls are system calls used to create, control, replace, wait for, and terminate processes. In POSIX, the main calls are `fork`, `waitpid`, `execve`, and `exit`.

1. Main Process Management Calls

Important Calls

The first major group of POSIX system calls deals with process management:

- `fork()` — create a new child process
- `waitpid()` — wait for a child to terminate
- `execve()` — replace the process image
- `exit()` — terminate a process

2. `fork()` System Call

Meaning of `fork`

`fork()` is the only standard way in POSIX to create a new process. It creates an almost exact duplicate of the calling process.

What `fork()` Copies

After `fork()`, the child gets a copy of:

- file descriptors,
- registers,
- variables,
- process state,
- almost the whole process image.

Parent and Child

After the call, two processes exist:

- the **parent process**,
- the **child process**.

They continue execution separately.

Important Note

At the moment of the fork, parent and child have identical variable values. But afterward, changes in one process do not affect the other, because the data are copied.

Shared Part

The **program text** or code segment is unchanged, so it is shared between parent and child.

3. Return Value of fork()

Return Values

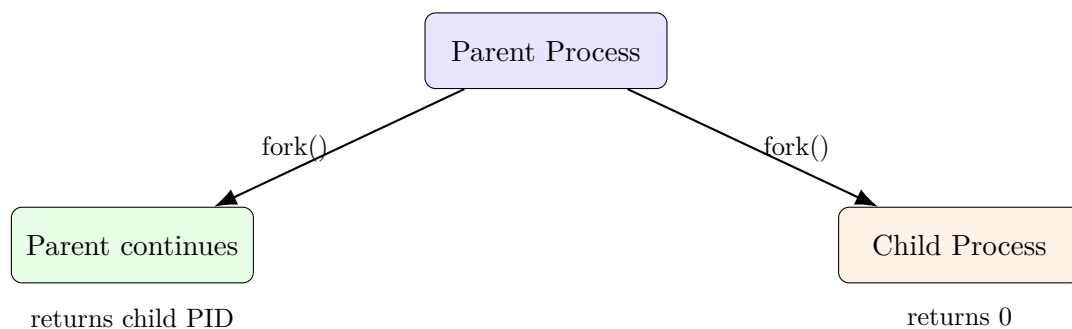
The fork() call returns different values in the two processes:

- in the **child**, it returns 0
- in the **parent**, it returns the child's **PID**

PID

PID means **Process Identifier**. Using the return value, each process can determine whether it is the parent or the child.

3.1 Simple Fork Diagram



4. waitpid() System Call

Meaning of waitpid

waitpid() makes a process wait until one of its child processes finishes execution.

Main Use

Usually the parent uses `waitpid()` after creating a child with `fork()`.

Form

```
pid = waitpid(pid, &statloc, options);
```

Functions of Parameters

- first parameter: which child to wait for
- second parameter: where exit status will be stored
- third parameter: options controlling the behavior

Special Case

If the first parameter is `-1`, then `waitpid()` can wait for **any child process**.

Status Information

When the child terminates, the value stored in `statloc` tells whether termination was:

- normal, or
- abnormal,

and may also contain the exit value.

5. `execve()` System Call

Meaning of `execve`

In most cases, after `fork()`, the child must run a different program. This is done by `execve()`, which replaces the child's current process image with a new program.

Main Idea

`execve()` does not create a new process. Instead, it transforms the existing process into a new program.

Form

```
s = execve(name, argv, environp);
```

Parameters of `execve`

- `name` — name of the program file to execute
- `argv` — pointer to argument array
- `environp` — pointer to environment array

Note

Several library routines such as `execl`, `execv`, `execle`, and `execve` are commonly used. In general discussion, they are often treated together as `exec`.

6. How Shell Uses `fork`, `waitpid`, and `execve`

Shell Behavior

The shell gives a practical example of process-management calls.

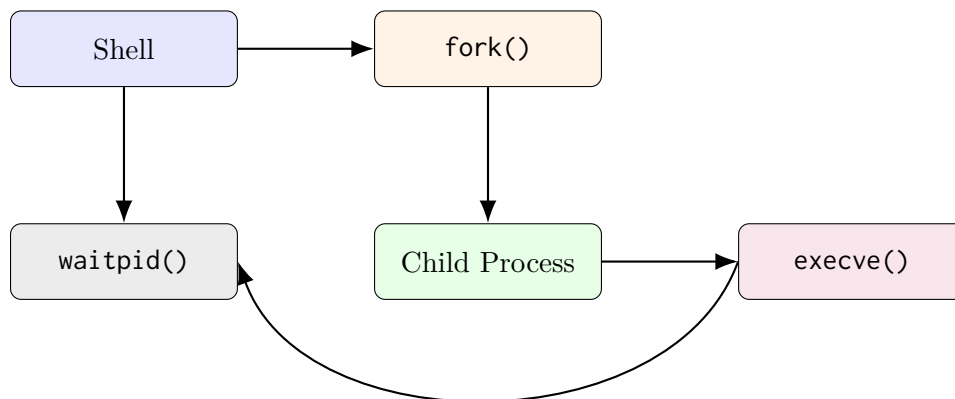
Steps in the Shell

1. shell reads a command,
2. shell calls `fork()`,
3. child process runs the command using `execve()`,
4. parent waits using `waitpid()`,
5. after child finishes, shell reads the next command.

Simplified Shell Code

```
#define TRUE 1
while (TRUE) {
    type_prompt();
    read_command(command, parameters);
    if (fork() != 0) {
        waitpid(-1, &status, 0);
    } else {
        execve(command, parameters, 0);
    }
}
```

6.1 Shell Process Flow Diagram



7. Command-Line Arguments

Example Command

Consider the command:

```
cp file1 file2
```

After `fork()`, the child uses `execve()` to run the program `cp` and passes the source and target file names as arguments.

Main Parameters in C

Most C programs have a main function like:

```
main(argc, argv, envp)
```

where:

- `argc` = number of command-line items
- `argv` = array of command-line strings
- `envp` = environment strings

For `cp file1 file2`

- `argc` = 3
- `argv[0]` = "cp"
- `argv[1]` = "file1"
- `argv[2]` = "file2"

8. Environment Array

Environment

The third parameter, `envp`, points to an array of strings of the form:

`name = value`

These strings pass environment information to programs.

Examples of Environment Information

Environment variables may store:

- terminal type,
- home directory,
- default printer,
- user preferences.

9. `exit()` System Call

Meaning of `exit`

When a process finishes execution, it uses `exit()` to terminate itself.

Form

```
exit(status);
```

Exit Status

The `status` value usually lies between 0 and 255. This value is returned to the parent through `statloc` in `waitpid()`.

10. Process Memory Segments

Memory Layout

In UNIX, a process is divided into three main memory segments:

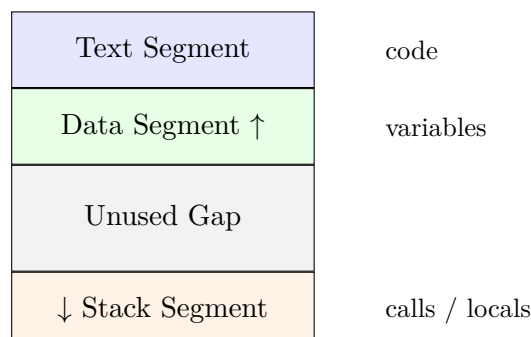
- **text segment** — program code
- **data segment** — variables
- **stack segment** — function calls and local data

Growth Direction

- the **data segment** grows upward
- the **stack segment** grows downward

Between them there is usually an unused gap of address space.

10.1 Simple Segment Diagram



Extra Note

The stack grows automatically as needed. But growth of the data segment is traditionally done explicitly using the `brk` system call. In practice, programmers usually use `malloc()` instead of calling `brk` directly.

System Calls for File Management

Definition

File-management system calls are system calls used to create, open, read, write, reposition, inspect, and close files. They provide the basic interface between programs and the file system.

1. Basic Idea

Main Concept

Many system calls are related to the file system. Some operate on **individual files**, while others work on **directories** or the **file system as a whole**.

This Section

Here the focus is on system calls that operate on **individual files**.

2. Opening a File

Open

Before a file can be read or written, it must first be **opened**. The **open** call specifies:

- the file name,
- the access mode,
- and optionally file-creation behavior.

File Name

A file may be opened using:

- an **absolute path name**, or
- a path relative to the **working directory**.

Access Modes

Common access modes are:

- `O_RDONLY` — open for reading only
- `O_WRONLY` — open for writing only
- `O_RDWR` — open for both reading and writing

Creating a File

To create a new file while opening, the parameter `O_CREAT` is used.

3. File Descriptor

Definition

When a file is opened successfully, the operating system returns a small integer called a **file descriptor**.

Use of File Descriptor

This file descriptor is then used in later operations such as:

- reading,
- writing,
- repositioning,
- closing the file.

4. Closing a File

Close

After file operations are finished, the file should be closed using `close()`.

Why Close is Important

Closing a file releases the file descriptor so that it may be reused by a later `open()` call.

5. Reading and Writing Files

Most Common Calls

The most heavily used file-management system calls are:

- `read`
- `write`

read Example

```
count = read(fd, buffer, nbytes);
```

write Example

```
count = write(fd, buffer, nbytes);
```

Idea

Both read and write use similar parameters:

- file descriptor,
- buffer address,
- number of bytes.

6. Sequential and Random Access

Sequential Access

Most programs read and write files **sequentially**. This means data are processed in order, one part after another.

Random Access

Some programs need **random access**, meaning they must jump directly to any location inside the file instead of moving only in sequence.

7. File Position Pointer

Position Pointer

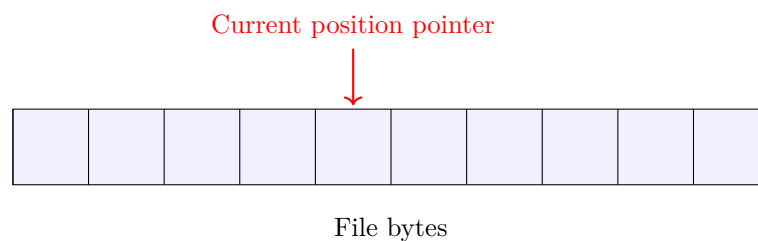
Each open file has an associated **position pointer**. It indicates the current location in the file.

Meaning

During sequential reading or writing, the position pointer usually points to:

- the next byte to be read, or
- the next byte to be written.

7.1 Simple Pointer Diagram



8. lseek() System Call

Meaning of lseek

The `lseek()` call changes the value of the file position pointer. It allows later `read()` or `write()` calls to start at a different place in the file.

General Form

```
pos = lseek(fd, offset, whence);
```

Parameters of lseek

- first parameter — file descriptor
- second parameter — desired file position / offset
- third parameter — reference point for the offset

Reference Point

The third parameter tells whether the offset is measured relative to:

- the **beginning** of the file,
- the **current position**,
- or the **end** of the file.

Return Value

The value returned by `lseek()` is the **absolute position** in the file, measured in bytes, after the pointer has been changed.

9. File Information

Stored Information

For every file, UNIX stores information such as:

- file mode,
- file size,
- time of last modification,
- and other related details.

File Mode

The **file mode** tells what kind of file it is, such as:

- regular file,
- special file,
- directory.

10. stat() and fstat()

Meaning

Programs can inspect file information using:

- stat()
- fstat()

stat()

stat() takes:

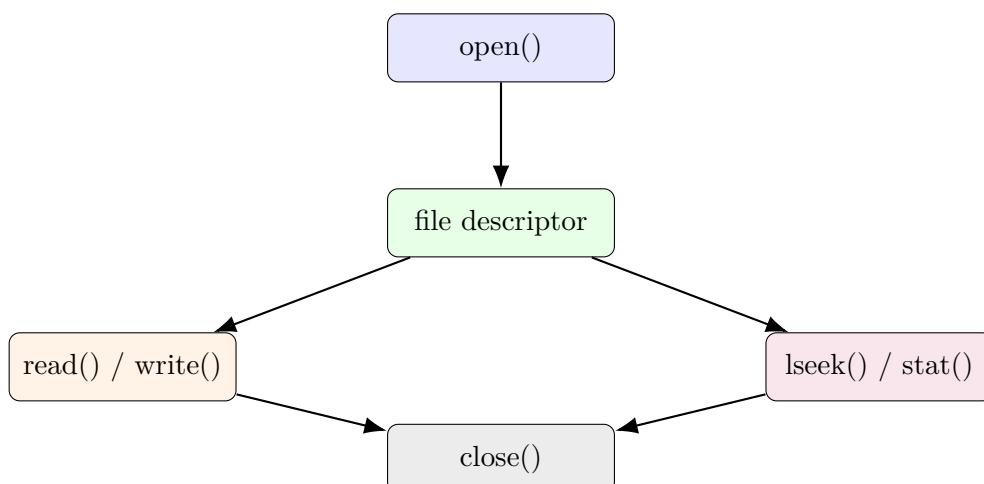
- a file name,
- and a pointer to a structure where the information will be stored.

fstat()

fstat() works in a similar way, but it is used for an **already open file**.

11. Flow of File Operations

11.1 Simple Flow Diagram



12. Main Calls at a Glance

Call	Purpose
<code>open()</code>	Open a file and obtain a file descriptor
<code>close()</code>	Close a file descriptor
<code>read()</code>	Read bytes from a file
<code>write()</code>	Write bytes to a file
<code>lseek()</code>	Change the file position pointer
<code>stat()</code>	Get information about a file by name
<code>fstat()</code>	Get information about an open file

13. Conclusion

Summary

System calls for file management allow programs to open files, read and write data, move to any position inside a file, inspect file information, and close files afterward. These calls form the basic operating-system support for working with files.

System Calls for Directory Management

Definition

Directory-management system calls are system calls used to create, remove, link, mount, and unmount directories or file systems. These calls deal with directories and the file system as a whole, rather than with only one file.

1. Basic Idea

Main Concept

Some system calls are mainly concerned with:

- directories,
- file-system structure,
- merging file systems,
- and shared file naming.

These are different from file-management calls that work on one specific file.

2. `mkdir()` and `rmdir()`

Creating and Removing Directories

The first two important directory-management calls are:

- `mkdir()` — creates a new directory
- `rmdir()` — removes an empty directory

Note

`rmdir()` removes only an **empty** directory.

3. `link()` System Call

Meaning of link

The `link()` system call allows the **same file** to appear under two or more names, possibly in different directories.

Purpose

A common use of `link()` is file sharing. For example, several members of a programming team may all use the same file from their own directories, even under different names.

Shared File vs Copy

A **shared file** is not the same as making private copies:

- In a shared file, there is only **one actual file**.
- Any change made by one user becomes visible immediately to others.
- In separate copies, later changes in one copy do not affect the others.

3.1 Example of `link()`

Example

```
link("/usr/jim/memo", "/usr/ast/note");
```

This creates a new directory entry so that:

/usr/jim/memo and */usr/ast/note*

refer to the same file.

4. How `link()` Works

i-number

Every file in UNIX has a unique number called an **i-number**. This identifies the file and points to its i-node entry.

Directory Structure

Conceptually, a directory is a collection of:

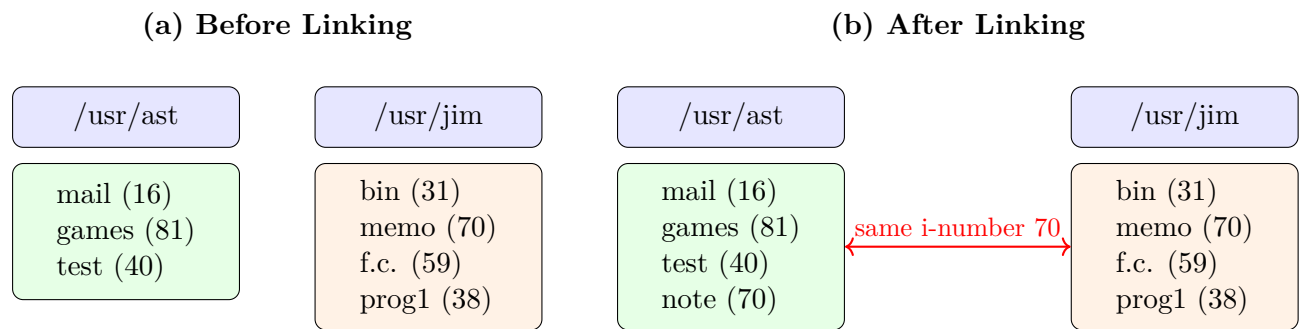
(i-number, ASCII name)

pairs.

What `link()` Actually Does

The `link()` call creates a **new directory entry** with a new name, but it uses the **same i-number** as an already existing file.

4.1 Before and After Linking



Result

After linking, two directory entries have the same i-number, so they refer to the same file.

5. unlink() System Call

Meaning of unlink

If one of the names of a linked file is removed using `unlink()`, the other name still remains.

When File is Really Deleted

UNIX removes the actual file from disk only when:

- all directory entries pointing to it are removed, and
- no links remain.

6. mount() System Call

Meaning of mount

The `mount()` system call allows two separate file systems to be merged into one single directory tree.

Typical Use

A common example is:

- main system files on one hard-disk partition,
- user files on another partition,
- USB drive inserted later and attached into the same tree.

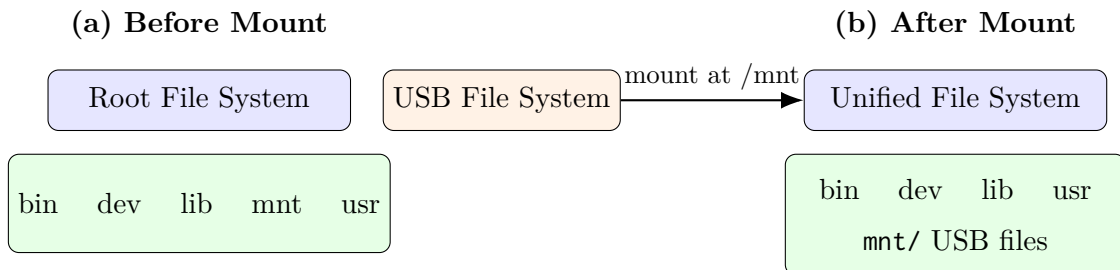
Example

```
mount("/dev/sdb0", "/mnt", 0);
```

Parameters of mount

- first parameter — block special file of the device
- second parameter — mount point in the directory tree
- third parameter — whether mounted read-write or read-only

6.1 Before and After Mounting



Main Advantage

After mounting, files on the new device can be accessed by normal path names from the root directory or working directory. The user does not need to worry about which physical device contains the file.

7. Integrated File Hierarchy

Single Tree

The `mount()` call makes it possible to integrate:

- removable media,
- hard-disk partitions,
- external hard disks,
- USB sticks

into one single file hierarchy.

Benefit

This gives a clean and device-independent way to access files.

8. `umount()` System Call

Meaning of `umount`

When a mounted file system is no longer needed, it can be detached from the file tree using the `umount()` system call.

Purpose

`umount()` safely removes the attached file system from the integrated hierarchy.

9. Main Calls at a Glance

Call	Purpose
<code>mkdir()</code>	Create a new directory
<code>rmdir()</code>	Remove an empty directory
<code>link()</code>	Create another name for the same file
<code>unlink()</code>	Remove a file name (directory entry)
<code>mount()</code>	Attach one file system to another
<code>umount()</code>	Detach a mounted file system

10. Conclusion

Summary

Directory-management system calls control the structure of directories and the file system as a whole. They allow directories to be created or removed, files to be shared through links, and multiple file systems to be combined into one integrated hierarchy through mounting.

Monolithic Systems

Definition

A **monolithic system** is an operating-system structure in which the entire operating system runs as one large program in **kernel mode**. All operating-system procedures are linked together into a single executable binary.

1. Basic Idea

Main Concept

In the monolithic approach:

- the whole operating system runs in kernel mode,
- it is built as one large executable program,
- all procedures are linked together,
- one procedure may call another directly.

Efficiency

This design is very efficient because any operating-system procedure can call any other useful procedure directly.

2. Main Characteristics

Procedure Organization

The operating system is written as a collection of procedures. These procedures are compiled and linked together to form one large object program.

Features

A monolithic system usually has:

- one large kernel,
- many procedures inside it,
- unrestricted procedure calls,
- high execution efficiency.

3. Advantages of Monolithic Systems

Advantages

Main advantages are:

- fast communication between procedures,
- simple direct procedure calls,
- efficient execution,
- low overhead.

4. Disadvantages of Monolithic Systems

Disadvantages

A monolithic system may also have important drawbacks:

- the system may become very large and hard to understand,
- thousands of procedures may interact without restriction,
- debugging becomes difficult,
- a crash in one procedure may crash the entire operating system.

5. Building the Monolithic Kernel

Construction

To build the operating system in this model:

1. all individual procedures or files are compiled,
2. all compiled parts are linked together,
3. the linker produces one executable kernel program.

Information Hiding

In a pure monolithic system, there is very little information hiding. Almost every procedure can see and call every other procedure.

Comparison

This is different from modular systems, where much of the internal information is hidden and only specific entry points are visible.

6. Internal Structure in a Monolithic System

Basic Structure

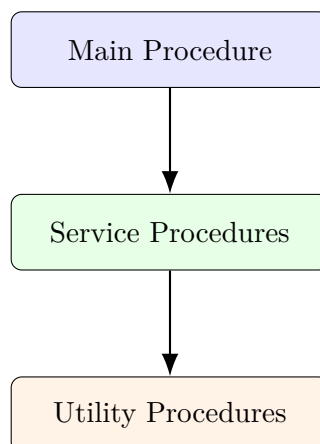
Even in a monolithic system, some internal organization is possible. A simple structure contains:

1. a **main program**,
2. a set of **service procedures**,
3. a set of **utility procedures**.

Roles

- The **main program** receives the request and invokes the correct service procedure.
- The **service procedures** perform the actual system calls.
- The **utility procedures** help the service procedures by doing common supporting work.

6.1 Simple Structure Diagram



7. System Call Handling

System Call Flow

In a monolithic system, a system call is requested by placing parameters in a known place, such as the stack, and then executing a **trap instruction**.

Steps

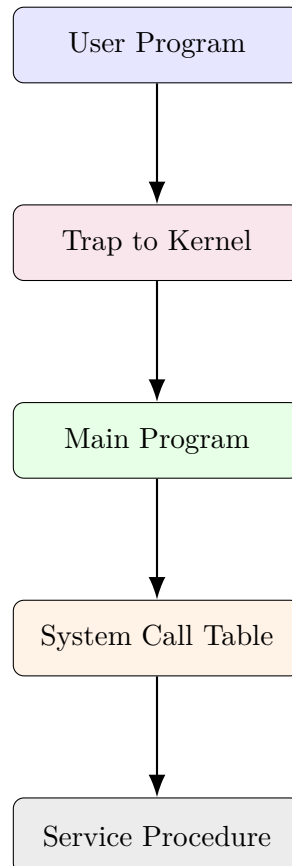
The basic sequence is:

1. user program places parameters,
2. trap instruction switches from user mode to kernel mode,
3. operating system checks which system call is requested,
4. system indexes into a table,
5. the correct service procedure is called.

Table Lookup

The operating system often uses a table where entry k contains a pointer to the procedure that handles system call k .

7.1 System Call Dispatch Diagram



8. Loadable Extensions

Extensions

In addition to the main kernel loaded at boot time, many operating systems support **loadable extensions**.

Examples

These extensions may include:

- I/O device drivers,
- file systems,
- additional kernel support components.

Loaded on Demand

These components are often loaded only when needed, rather than being permanently included in the core kernel image.

9. Shared Libraries and DLLs

Names in Different Systems

Different operating systems use different names for such loadable extensions:

- In UNIX, they are often called **shared libraries**.
- In Windows, they are called **DLLs** (Dynamic-Link Libraries).

Windows DLL

Windows DLL files usually have the extension:

.dll

10. Summary Points

Quick Review

A monolithic system:

- runs entirely in kernel mode,
- is built as one large executable,
- allows direct calls among procedures,
- is efficient but less safe and harder to manage,
- may still have internal structure through main, service, and utility procedures.

Layered Systems

Definition

A **layered system** is an operating-system structure in which the system is organized as a hierarchy of layers. Each layer is built on top of the layer below it and provides services to the layer above it.

1. Basic Idea

Main Concept

In the layered approach:

- the operating system is divided into several layers,
- each layer has its own function,
- each higher layer uses the services of the lower layer,
- this makes the system more organized and easier to understand.

Main Advantage

The layered design improves structure, clarity, and separation of responsibilities inside the operating system.

2. THE Operating System

Historical Example

The first important layered operating system was the **THE system**, built at the Technische Hogeschool Eindhoven in the Netherlands by E. W. Dijkstra and his students.

System Type

THE was a simple **batch system** built for the **Electrologica X8** computer.

Interesting Note

The machine had:

- 32K of 27-bit words,
- limited memory by modern standards,
- a design where careful organization was very important.

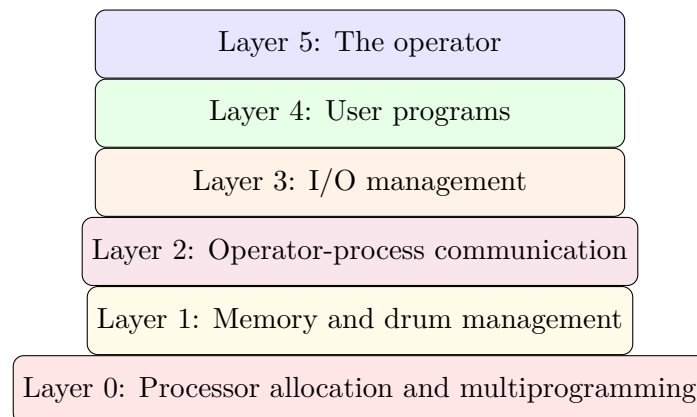
3. Six Layers of THE System

Layer Structure

The THE operating system was divided into **six layers**, numbered from 0 to 5.

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

3.1 Simple Layer Diagram



4. Layer 0

Processor Allocation

Layer 0 handled:

- allocation of the processor,
- switching between processes,
- response to interrupts and timer expiration.

Role

This layer provided the basic **multiprogramming** of the CPU. Above this layer, processes could behave as if each had its own processor.

5. Layer 1

Memory Management

Layer 1 managed:

- main memory,
- a 512K word drum,
- movement of process pages between them.

Main Idea

Above layer 1, a process did not need to worry about whether it was in main memory or on the drum. Layer 1 handled that automatically.

6. Layer 2

Operator Communication

Layer 2 handled communication between each process and the operator console.

Effect

Above this layer, each process effectively had its own operator console.

7. Layer 3

I/O Management

Layer 3 managed:

- I/O devices,
- buffering of input streams,
- buffering of output streams.

Abstraction

Above layer 3, processes worked with abstract I/O devices instead of dealing directly with real hardware devices and their peculiar behavior.

8. Layer 4

User Programs

Layer 4 contained the **user programs**.

Benefit

User programs at this level did not need to handle:

- processor management,
- memory management,
- operator console handling,
- or I/O device control.

9. Layer 5

Operator Layer

Layer 5 contained the **system operator process**.

10. Advantages of Layered Systems

Advantages

Layered systems provide:

- better organization,
- clearer structure,
- easier understanding,
- separation of responsibilities,
- simpler system design.

11. MULTICS and Ring Structure

Generalization

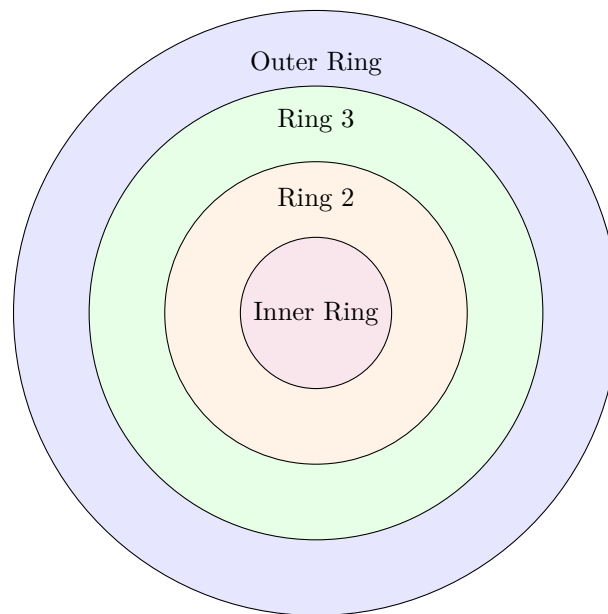
A further generalization of layering appeared in the **MULTICS** system. Instead of layers, MULTICS used a system of **concentric rings**.

Ring Idea

In MULTICS:

- inner rings were more privileged,
- outer rings were less privileged,
- outer-ring procedures needed special permission to call inner-ring procedures.

11.1 Ring Model Diagram



System Call Style

If a procedure in an outer ring wanted to call a procedure in an inner ring, it had to perform the equivalent of a **system call**. Its parameters were carefully checked before access was allowed.

12. Protection in MULTICS

Hardware Enforcement

In MULTICS, the ring mechanism was not just a design idea. It was actually enforced by the **hardware** at run time.

Memory Protection

Although the whole operating system was in the address space of each user process, the hardware could protect individual procedures or memory segments against:

- reading,
- writing,
- executing.

13. THE vs MULTICS

Important Difference

The layering of THE was mainly a **design aid**, because all parts were eventually linked into one executable program.

MULTICS Advantage

In MULTICS, the ring mechanism existed at **run time** and was enforced by hardware, making it stronger and more secure than simple design-time layering.

14. Use of Rings for User Subsystems

Extension of Ring Concept

The ring model could also be used to structure user subsystems, not just the operating system.

Example

A professor could run a grading program in ring n , while student programs ran in ring $n + 1$. This would prevent student programs from changing their grades.

15. Summary Points

Quick Review

A layered system:

- organizes the operating system into levels,
- lets each layer depend on the one below,
- improves structure and abstraction,
- was first demonstrated in the THE system,
- was further generalized by the ring model of MULTICS.

Microkernels

Definition

A **microkernel** is an operating-system design in which only a very small core runs in **kernel mode**, while most other operating-system components run as separate **user-mode processes**. The goal is to increase reliability, modularity, and fault isolation.

1. Basic Idea

Main Concept

In a microkernel design:

- the kernel is kept as small as possible,
- only essential functions remain in kernel mode,
- file systems, drivers, and many services run in user mode,
- faults in one component are less likely to crash the whole system.

Reason for the Design

Bugs inside the kernel are very dangerous because they can crash the entire operating system immediately. User processes have less power, so bugs there are usually less destructive.

2. Why Smaller Kernel is Safer

Bug Problem

Large software systems usually contain many bugs. Therefore, if the kernel is huge, the chance of serious kernel bugs also becomes high.

Effect

In a monolithic system:

- many drivers and subsystems run inside the kernel,
- one bad driver may crash the entire machine.

In a microkernel system:

- many components run as isolated user processes,
- failure of one component usually affects only that component.

Example

A buggy audio driver in a monolithic system may crash the whole computer. In a microkernel system, the same buggy audio driver may only stop or distort sound, while the rest of the system continues running.

3. Main Features of Microkernels

Core Characteristics

A microkernel system usually has:

- a very small kernel,
- separate user-mode drivers,
- separate user-mode file systems,
- message passing between processes,
- strong isolation and protection.

4. Common Use of Microkernels

Where They Are Used

Microkernels are especially common in systems that need very high reliability.

Typical Application Areas

They are widely used in:

- real-time systems,
- industrial systems,
- avionics,
- military systems,
- other mission-critical environments.

Examples

Some well-known microkernel-based systems include:

- Integrity
- K42
- L4
- PikeOS
- QNX
- Symbian
- MINIX 3

5. MINIX 3 as an Example

MINIX 3

MINIX 3 is a POSIX-conformant open-source microkernel system. It pushes modularity very far by moving most operating-system functionality into independent user-mode processes.

Microkernel Size

The MINIX 3 microkernel is very small compared with full monolithic kernels. Its main responsibilities include:

- process management,
- scheduling,
- interrupt handling,
- interprocess communication,
- a small set of kernel calls.

6. What the Microkernel Does

Microkernel Functions

The microkernel itself performs only the most essential tasks, such as:

- catching interrupts,
- switching processes,
- scheduling,
- passing messages between processes,
- handling low-level kernel calls.

Kernel Calls

These kernel calls support operations like:

- attaching interrupt handlers,
- moving data between address spaces,
- installing memory maps for new processes.

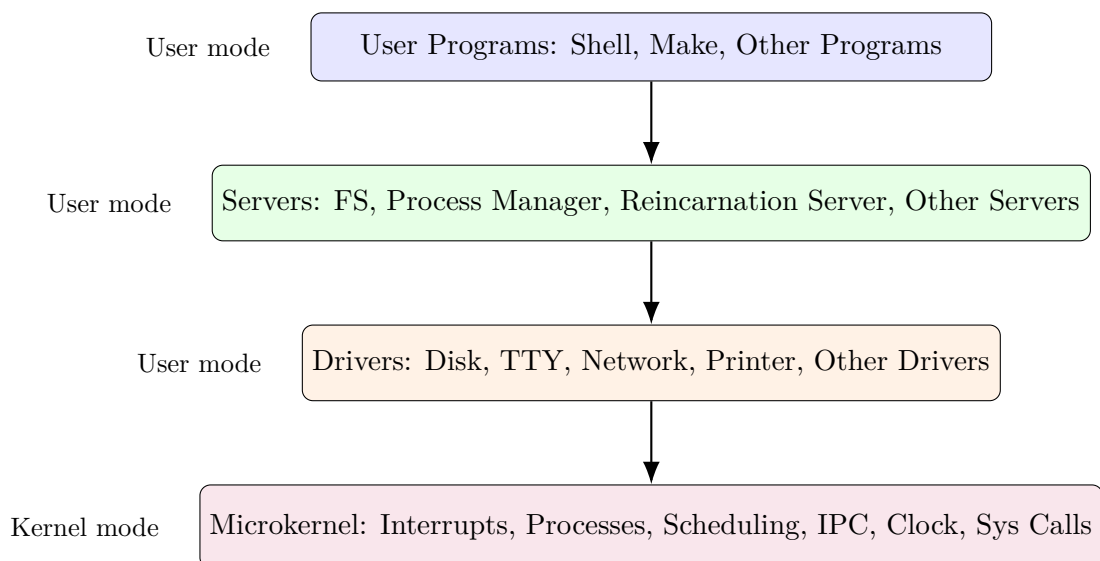
7. Process Structure of MINIX 3

Layered User-Mode Structure

Outside the microkernel, MINIX 3 has several user-mode layers:

- device drivers,
- servers,
- user programs.

7.1 Simplified MINIX 3 Structure



8. Device Drivers in User Mode

Drivers

In MINIX 3, most device drivers run as separate user-mode processes, not inside the kernel.

Restriction

Because drivers run in user mode:

- they cannot directly access hardware I/O ports,
- they must ask the kernel to perform low-level I/O operations,
- the kernel checks whether the driver is authorized.

Benefit

This protection prevents one buggy driver from damaging unrelated parts of the system. For example, a faulty audio driver cannot accidentally write on the disk.

9. Servers in User Mode

Servers

Above the drivers is a layer of user-mode **servers**. These servers perform most of the higher-level work of the operating system.

Examples of Servers

Examples include:

- file servers for file systems,
- the process manager,
- the reincarnation server,
- other service processes.

Service Request

If a user process wants to perform a POSIX operation such as **read**, it sends a short message to the appropriate server asking for that service.

10. Interprocess Communication

IPC

A central mechanism in microkernels is **interprocess communication (IPC)**. Processes cooperate by sending messages to each other.

Why IPC Matters

Since drivers, servers, and user programs are separated into different processes, message passing is needed to:

- request services,
- transfer information,
- coordinate system actions.

11. Reincarnation Server

Special Server

One important MINIX 3 server is the **reincarnation server**.

Its Job

The reincarnation server:

- monitors drivers and servers,
- checks whether they are functioning correctly,
- restarts faulty components automatically.

Self-Healing

Because of this mechanism, the system can recover from some failures automatically, without user intervention. This improves reliability and makes the system **self-healing**.

12. Restrictions and Protection

Controlled Privileges

Microkernel systems place strict limits on what each process can do.

Examples of Restrictions

Restrictions may control:

- which I/O ports a driver may access,
- which kernel calls a process may use,
- which processes may send messages to others,
- when one process may access another process's address space.

Result

Each component gets only the power required for its own work and nothing more. This greatly reduces the damage that a buggy component can cause.

13. Policy and Mechanism

Important Design Principle

A related idea is to keep the **mechanism** in the kernel, but move the **policy** outside the kernel whenever possible.

Meaning

For example, in process scheduling:

- the **mechanism** is the actual act of selecting and running a process,
- the **policy** is the rule for deciding which process should get priority.

Scheduling Example

The kernel may provide the mechanism to run the highest-priority runnable process. A user-mode component may provide the policy by deciding what priorities the processes should have.

Advantage

Separating policy from mechanism makes the kernel smaller, cleaner, and more flexible.

14. Advantages of Microkernels

Main Advantages

Microkernels provide:

- higher reliability,
- better fault isolation,
- stronger modularity,
- easier recovery from component failures,
- improved security and protection.

15. Possible Drawback

Possible Limitation

Because many services are moved out of the kernel and communicate through messages, microkernels may involve more communication overhead than monolithic kernels.

16. Summary Points

Quick Review

A microkernel:

- keeps only essential functions in kernel mode,
- moves many services to user mode,
- uses message passing for communication,
- isolates faults better than monolithic systems,
- is especially useful in high-reliability systems.

Client–Server Model

Definition

The **client–server model** is an operating-system and system-design model in which one group of processes, called **servers**, provides services, while another group of processes, called **clients**, requests and uses those services.

1. Basic Idea

Main Concept

In the client–server model:

- **clients** ask for services,
- **servers** perform those services,
- communication usually happens through **message passing**.

Relation with Microkernel

The lowest layer is often a **microkernel**, but that is not required. The main idea of this model is simply the existence of client processes and server processes.

2. Clients and Servers

Client

A **client** is a process that requests a service from another process.

Server

A **server** is a process that provides a specific service to clients.

Examples of Services

A server may provide services such as:

- file service,
- process management,
- terminal service,
- web service,
- database service.

3. Communication by Message Passing

Message Passing

Communication between clients and servers is usually done through **message passing**.

How It Works

To get a service:

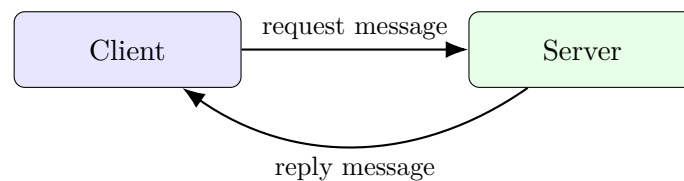
1. the client prepares a message describing what it wants,
2. the client sends the message to the server,
3. the server performs the work,
4. the server sends back a reply.

Key Idea

Whether the client and server are on the same machine or different machines, the basic idea is the same:

request → service → reply

3.1 Simple Request–Reply Diagram



4. Same Machine or Different Machines

Local and Remote Use

The client–server model can be used:

- on a **single machine**,
- or across a **network of machines**.

Local Case

If the client and server are on the same machine, some optimizations are possible. However, conceptually they still communicate by message passing.

Remote Case

If the client and server are on different computers, the messages travel through a local-area network or a wide-area network.

5. Transparency of the Model

Transparency

A major strength of the client–server model is that the client often does not need to know whether the server is:

- local on the same machine, or
- remote on another machine.

Why This Matters

From the client’s point of view, the same basic operation happens in both cases:

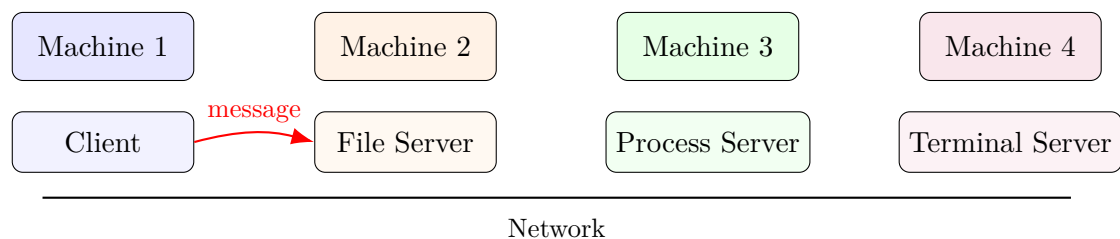
- send request,
- wait,
- receive reply.

6. Client–Server Over a Network

Network Version

The client–server idea can be extended naturally to a distributed system in which clients and servers run on different computers connected by a network.

6.1 Network Diagram



7. Abstraction of the Model

Abstraction

The client–server model is an abstraction that works in both situations:

- one computer,
- many networked computers.

So the same idea can be used for local operating-system structure or for distributed systems.

8. Real-Life Use

Modern Usage

Today, many systems use home computers, laptops, or phones as clients and more powerful machines elsewhere as servers.

Web Example

A common example is the Web:

- a PC or phone sends a request for a web page,
- the web server receives the request,
- the server sends the page back.

This is a classic example of the client–server model over a network.

9. Advantages of the Client–Server Model

Advantages

The client–server model offers:

- clear separation between service provider and service user,
- modular design,
- easier distribution over networks,
- flexibility in placing services,
- easier system expansion.

10. Summary Points

Quick Review

In the client–server model:

- clients request services,
- servers provide services,
- communication is usually by message passing,
- the model works on one machine or across many machines,
- the Web is a common real-world example.

Virtual Machines

Definition

A **virtual machine** is a software-created machine that behaves like real hardware. It allows one physical computer to run multiple independent operating systems or environments. Based on user text :contentReference[oaicite:0]index=0

1. Basic Idea

Main Concept

Virtualization separates two important functions:

- **multiprogramming**, and
- providing a more convenient **extended machine**.

A virtual-machine system gives several virtual machines, each behaving like a real machine.

Key Observation

Each virtual machine can act like a complete copy of the bare hardware, including:

- kernel/user mode,
- I/O,
- interrupts,
- and other hardware features.

2. VM/370

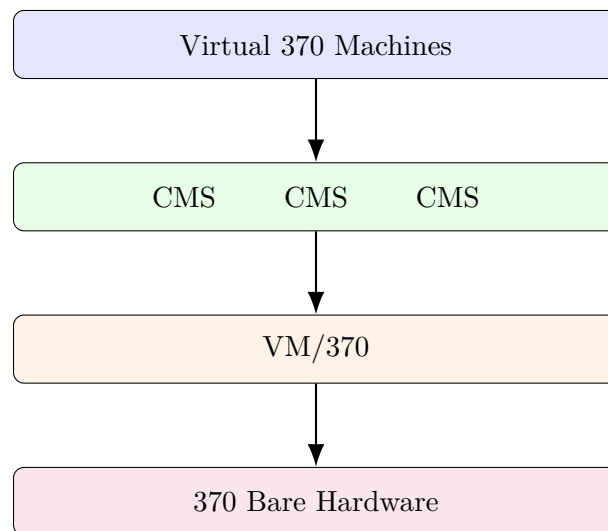
Historical Example

An important early virtual-machine system was IBM's **VM/370**, originally called **CP/CMS**. It became a major virtualization system for IBM mainframes.

Main Idea of VM/370

The heart of VM/370 was the **virtual machine monitor**, which ran directly on bare hardware and created multiple virtual machines above it.

2.1 VM/370 Structure



3. Why Virtual Machines Were Useful

Benefit

Since each virtual machine looked like a real machine, each one could run any operating system that could run on the actual hardware.

Examples

Different virtual machines could run:

- OS/360,
- transaction-processing systems,
- or CMS for interactive users.

Important Result

Different virtual machines on the same physical computer could run **different operating systems at the same time**.

4. How a System Call Worked in VM/370

Behavior

When a program running inside CMS made a system call, it was trapped to the operating system inside that **same virtual machine**, not directly to VM/370.

I/O Handling

When the guest operating system later issued hardware I/O instructions, those instructions were trapped by VM/370, which simulated the real hardware behavior.

5. Why Virtualization Became Popular Again

Rediscovery

Although virtualization existed for decades on mainframes, it became especially important again in the PC and server world because of new practical needs.

Major Reasons

Important reasons include:

- running multiple servers on one machine,
- isolating failures between services,
- reducing hosting cost,
- running multiple operating systems together,
- improving flexibility for users and companies.

6. Server Consolidation

title

Many organizations used to run mail servers, web servers, FTP servers, and other services on separate machines. Virtualization allows all of them to run on one machine while remaining separated.

Advantage

If one virtual server crashes, the others can continue running. This improves isolation and saves hardware cost.

7. Web Hosting Use

Virtual Hosting

Virtual machines are popular in web hosting because one physical machine can host many independent virtual machines.

Benefit to Customers

A customer renting a virtual machine can:

- choose their own operating system,
- install their own software,
- get more freedom than shared hosting,
- pay much less than for a dedicated server.

8. Running Multiple OS on One Computer

End-User Use

Virtualization is also useful for users who want to run more than one operating system at the same time, such as Windows and Linux on one machine.

9. Hypervisors

Hypervisor

A **hypervisor** is the software layer that creates and manages virtual machines. The older name **virtual machine monitor** is still also used.

9.1 Type 1 and Type 2 Hypervisors

Type 1 Hypervisor

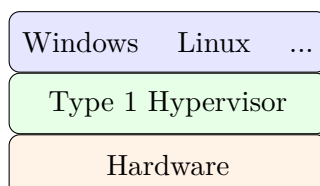
A **type 1 hypervisor** runs directly on the hardware and manages virtual machines without relying on a host operating system.

Type 2 Hypervisor

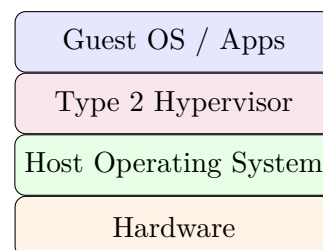
A **type 2 hypervisor** runs on top of a host operating system and uses the host system for services such as file storage and process creation.

9.2 Simple Hypervisor Comparison

(a) Type 1 Hypervisor



(b) Type 2 Hypervisor



10. Virtualizability Problem

CPU Requirement

To run virtual-machine software efficiently, the CPU should be **virtualizable**. This means privileged instructions must trap correctly so the hypervisor can handle them.

Old Problem

On some older processors, privileged instructions executed in user mode were simply ignored instead of trapping. This made efficient virtualization very difficult.

11. Machine Simulators and Binary Translation

Machine Simulator

One solution was to use a **machine simulator**, which translates and runs machine instructions in software.

Binary Translation

Some systems improved performance by translating blocks of code on the fly, storing them in a cache, and reusing them later. This technique is called **binary translation**.

Performance

Binary translation improved performance compared with simple interpreters, but it was still often not fast enough for demanding commercial use.

12. Hybrid Type 2 Hypervisors

Practical Improvement

Performance improved further when type 2 hypervisors added a kernel module to help with low-level work.

Result

Modern commercial type 2 hypervisors often use this hybrid design: part runs in user mode and part runs with kernel assistance.

13. Examples of Popular Hypervisors

Examples

Popular hypervisors include:

- VMware Workstation
- Xen
- KVM
- VirtualBox
- Hyper-V

14. Type 1 vs Type 2 Difference

Main Practical Difference

The main difference is:

- a type 2 hypervisor uses a **host operating system**,
- a type 1 hypervisor does **not** depend on a host operating system.

Storage Example

A type 2 hypervisor can store a virtual disk as a file inside the host operating system's file system.

A type 1 hypervisor must manage storage itself, usually on a raw disk partition.

15. Guest Operating System

Guest OS

The operating system running inside a virtual machine is called the **guest operating system**.

User Experience

After booting, the guest operating system behaves much like it would on real hardware, even though it is actually running inside a virtual environment.

16. Paravirtualization

Paravirtualization

A different approach is **paravirtualization**, where the operating system is modified to avoid problematic control instructions instead of simulating a completely unchanged machine.

Difference

Paravirtualization is related to virtualization, but it is not the same as true full virtualization.

17. Java Virtual Machine

JVM

Another important virtual machine is the **Java Virtual Machine (JVM)**. It is used to run Java programs in a portable way.

How It Works

- Java source code is compiled into JVM code,
- the JVM interpreter or runtime executes that code,
- the same JVM program can run on many different platforms.

Main Advantage

Because Java programs target the JVM instead of a specific hardware CPU, they can be distributed and run on many different machines more easily.

Security Advantage

If implemented properly, the JVM can check incoming programs for safety and execute them in a protected environment.

18. Advantages of Virtual Machines

Main Advantages

Virtual machines provide:

- isolation between systems,
- better hardware utilization,
- support for multiple operating systems,
- flexibility in hosting and testing,
- portability in systems like JVM.

19. Summary Points

Quick Review

Virtual machines:

- simulate complete computers,
- are managed by hypervisors,
- may run multiple guest operating systems,
- are useful in servers, hosting, and personal computing,
- also appear in other forms such as the JVM.